IQRA UNIVERSITY IU

Islamabad Campus

Practical Workbook

CSC 223-L

Computer Architecture Lab

x86 (emu8086) | MIPS (QtSPIM) | BS (Computer Science) | Fall 2026

Student Name _____

Registration No. _____

Section _____

Instructor: Faiza Ali, Junior Lecturer
faiza.ali@iqraisb.edu.pk | Office A8

DEPARTMENT OF COMPUTER SCIENCE
Faculty of Engineering Sciences and Technology (FEST)
Iqra University, Islamabad

INTRODUCTION

A course on Computer Architecture is meant to provide insight into the working of computer systems. Writing machine-dependent software such as compilers, operating systems and device drivers needs knowledge of the structural and functional organisation of computers. A programmer interested in high performance studies computer architecture to write programs that gain maximum performance from a given machine, and an engineer needs it to understand the cost/performance trade-offs that result from design decisions.

This practical workbook strengthens the topics covered in the theory course and follows the HEC-aligned course outline of **CSC 223-L Computer Architecture Lab** at Iqra University. It has two parts:

- **Part I (Labs 1–8)** – assembly language programming of the Intel x86 family (CISC) using the **emu8086** emulator. emu8086 shows registers, flags and memory while a program runs instruction by instruction, which makes it ideal for learning.
- **Part II (Labs 9–13)** – assembly language of the **MIPS** processor (RISC), used widely in embedded systems, using the **QtSPIM** simulator, followed by simulations of cache memory and virtual-memory address translation.

Students therefore learn the assembly language of both a CISC (x86) and a RISC (MIPS) machine and can compare the two design philosophies. Every lab session is designed for **3 hours** and contains an object, theory, worked example programs (all tested), a step-by-step procedure and exercises that are graded on the lab rubrics. The sessions are intended to be thought-provoking so that students develop their own ways of solving problems.

PREPARED BY

This workbook has been prepared by **Faiza Ali**, Junior Lecturer, Department of Computer Science, Iqra University Islamabad (faiza.ali@iqraisb.edu.pk, Office A8), for the Fall 2026 semester.

GENERAL INSTRUCTIONS

1. Bring this workbook to every lab session and keep it until the end of the semester.
2. Read the theory and example programs of a session **before** coming to the lab.
3. Complete the exercises during the session and get them checked by the instructor before leaving.
4. Save programs as `LabXX_TaskY_RegNo.asm` and upload the programs and screenshots to the LMS before the deadline.
5. Write neat, indented and commented code. Every program must start with a comment giving its purpose.
6. Plagiarism is strictly forbidden. Copied work receives zero marks in all rubrics of that lab and no resubmission is allowed.
7. Late submissions will be penalised as per departmental policy.
8. Food and drinks are not allowed in the lab. Shut down the computer properly at the end of the session.

VERSION HISTORY

Date	Updated By	Details
October 2026	Faiza Ali (Junior Lecturer)	First version: 13 lab sessions aligned with the CSC 223-L course outline (emu8086 for x86, QtSPIM for MIPS), lab rubrics and Open-Ended Lab.

COURSE INFORMATION

1. Basic Information

Course Code	CSC 223-L	**Course Name**	Computer Architecture Lab
Credit Hours	2+1	**Prerequisite(s)**	CSC 222
Instructor	Faiza Ali	**Designation**	Junior Lecturer
Email	faiza.ali@iqraisb.edu.pk	**Semester**	Fall 2026
Office Location	A8	**Office Hours**	As announced on LMS

2. Course Objective

The course aims to develop a clear understanding of computer architecture principles and their importance in modern computing systems. It seeks to enhance students' ability to analyze system performance and design efficient architectures. Students will also gain insights into advanced architectural concepts for optimized system design.

3. Course Contents

The course covers the fundamentals of computer architecture, including instruction set architectures, performance metrics, and basic processor design. Topics include memory hierarchies, cache design, virtual memory, pipeline processing, and hazard resolution techniques. The study of I/O systems, DMA, interrupts, and parallel processing introduces students to system-level interactions. Advanced topics, such as RISC and CISC architectures, are also included.

4. Course Learning Outcomes

CLO	CLO Statement	BT Level	GA	ACM KA	SDG	% Weight
CLO1	Apply concepts of instruction set architecture and assembly language programming to develop solutions for x86 and MIPS systems.	P3	GA2	#2 AR	4	65%
CLO2	Analyze and implement system-level operations, such as procedures, macros, and memory management, using assembly language and simulators.	A3	GA6			35%

Note: On successful completion of the course, GA1 (Academic Education) will automatically attain.

5. Assessment (Lab)

Assessment	CLO	Weight
Lab Manual (lab tasks)	CLO1/2	20%
Midterm Exam	CLO1	25%
Project / Open-Ended Lab	CLO1/2	15%
Final Exam	CLO1/2	40%

Weights as given in Section 6 (CLO Outcome-Based Assessment) of the course outline. Every CLO is assessed at least three times.

LAB RUBRICS

Every lab session is assessed on five criteria (R1–R5), each marked from 0 to 3, i.e. 15 marks per lab. **Group 1** rubrics are used for individual lab sessions and **Group 2** rubrics for the Open-Ended Lab / Project.

Lab Rubrics (Group 1) – Individual Lab Sessions

Criteria	Unacceptable (Marks = 0)	Substandard (Marks = 1)	Adequate (Marks = 2)	Proficient (Marks = 3)
R1 Completeness and Accuracy	The program failed to produce the right, accurate result.	The program produced inaccurate or incomplete results; not all features were implemented.	The program was correctly functional and most of the features were implemented.	The program was correctly functional and all the features were implemented.
R2 Syntax and Semantics	The student fails to find the syntax and semantic errors in the program.	The student finds a few syntax and semantic errors with extensive guidance.	The student finds most of the syntax and semantic errors with minimum guidance.	The student finds all syntax and semantic errors without any guidance.
R3 Demonstration	The student failed to demonstrate a clear understanding of the assigned task.	Basic understanding, but asked questions were not answered.	Basic understanding; fundamental answers to the asked questions.	Accurate understanding of the lab objective and concepts; all questions answered completely and correctly.
R4 Complexity and Readability	The code is poorly organised and very difficult to read.	The code is readable only by someone who knows what it is supposed to do.	The code is fairly easy to read.	The code is exceptionally well organised, commented and easy to follow.
R5 Perseverance and Plagiarism	The complete program is copied; results in zero for all rubrics.	Most of the working program is copied; minor contribution by the student.	Most of the program is contributed by the student; minor copied components.	The complete working program is contributed by the student.

Lab Rubrics (Group 2) – Open-Ended Lab / Project

Criteria	Unacceptable (Marks = 0)	Substandard (Marks = 1)	Adequate (Marks = 2)	Proficient (Marks = 3)
R1 Problem Analysis and Design	No clear analysis of the problem; design missing.	Partial analysis; important design elements missing or incorrect.	Correct analysis and design with minor gaps.	Complete, well-justified design; alternatives (e.g. x86 vs MIPS) compared.
R2 Implementation and Results	The system failed to produce the right result.	The system produced inaccurate or incomplete results.	The system was correctly functional and most features were implemented.	The system was correctly functional, all features implemented and results analysed.
R3 Demonstration and Viva	Failed to demonstrate a clear understanding of the task.	Basic understanding, but questions were not answered.	Moderate understanding; answers to questions are basic.	Accurate understanding; all questions answered completely and correctly.
R4 Contribution / Group Participation	Little commitment to group goals; fails to perform assigned roles.	Commitment to group goals but difficulty performing assigned roles.	Commitment to group goals; carries out assigned roles effectively.	Actively helps to identify group goals and works effectively in all roles.
R5 Report and Presentation	Poor report and presentation; technical terms lacking.	Report/presentation lacks clarity; little use of technical terms.	Acceptable report and presentation; adequate use of technical terms.	Well-organised, clear report and presentation; good technical vocabulary.

WEEKLY PLAN AND MAPPING OF LAB SESSIONS

Week No.	Lab No.	Lab Description	Software	CLO
1	01	Exploring Instruction Set Architecture (ISA) of x86 Machines	emu8086	1-2
2	02	Learning to program in Assembly Language of x86 Machines	emu8086	1-2
3	03	Using MACROS for Input / Output and Data Conversion	emu8086	1-2
4	04	Using x86 Data Transfer and Arithmetic Instructions	emu8086	1-2
5	05	Implementing Branching in x86 Assembly Language	emu8086	1-2
6	06	Implementation of Loop Structures in x86 Assembly Language	emu8086	1-2
7	–	Open Ended Lab / Project Assigned	–	1-2
8	–	Midterm Exam	–	–
9	07	Array Processing in x86 Assembly Language	emu8086	1-2
10	08	Development of Procedures and Macros in x86 Assembly Language	emu8086	1-2
11	09	Familiarization with SPIM – a MIPS simulator	QtSPIM	1-2
12	10	SPIM Console, System Calls and Procedures in MIPS Assembly Language	QtSPIM	1-2
13	11	Vector Operations in MIPS Assembly and Loop Unrolling	QtSPIM	1-2
14	12	Simulating Cache Read/Write Operations	QtSPIM	1-2
15	13	Address Translation in a Virtual Memory System	QtSPIM	1-2
16	–	Open Ended Lab / Project Assessment	–	1-2
17	–	Final Exam	–	1-2

Contact hours: 3 per lab session. Rubric Group 1 applies to Labs 1–13; Rubric Group 2 applies to the Open-Ended Lab / Project.

MARKS SHEET

Date	Lab Session	Max	R1	R2	R3	R4	R5	Instructor Sign
	Lab 01: Exploring Instruction Set Architecture (ISA) of x86 Machines	15						
	Lab 02: Learning to program in Assembly Language of x86 Machines	15						
	Lab 03: Using MACROS for Input / Output and Data Conversion	15						
	Lab 04: Using x86 Data Transfer and Arithmetic Instructions	15						
	Lab 05: Implementing Branching in x86 Assembly Language	15						
	Lab 06: Implementation of Loop Structures in x86 Assembly Language	15						
	Lab 07: Array Processing in x86 Assembly Language	15						
	Lab 08: Development of Procedures and Macros in x86 Assembly Language	15						
	Lab 09: Familiarization with SPIM – a MIPS simulator	15						
	Lab 10: SPIM Console, System Calls and Procedures in MIPS Assembly Language	15						
	Lab 11: Vector Operations in MIPS Assembly and Loop Unrolling	15						
	Lab 12: Simulating Cache Read/Write Operations	15						
	Lab 13: Address Translation in a Virtual Memory System	15						
	Open-Ended Lab / Project	15						
Grand Total		**210**						

CONTENTS

Lab Session No.	Object	Week
01	Exploring Instruction Set Architecture (ISA) of x86 Machines	1
02	Learning to program in Assembly Language of x86 Machines	2
03	Using MACROS for Input / Output and Data Conversion	3
04	Using x86 Data Transfer and Arithmetic Instructions	4
05	Implementing Branching in x86 Assembly Language	5
06	Implementation of Loop Structures in x86 Assembly Language	6
07	Array Processing in x86 Assembly Language	9
08	Development of Procedures and Macros in x86 Assembly Language	10
09	Familiarization with SPIM – a MIPS simulator	11
10	SPIM Console, System Calls and Procedures in MIPS Assembly Language	12
11	Vector Operations in MIPS Assembly and Loop Unrolling	13
12	Simulating Cache Read/Write Operations	14
13	Address Translation in a Virtual Memory System	15
OEL	Open-Ended Lab / Project: CISC vs RISC Comparative Mini-Project	7 / 16
App. A	DOS INT 21h services, ASCII codes and x86 instruction summary	–
App. B	MIPS registers, instructions and SPIM system calls	–

LAB SOFTWARE SETUP

emu8086 – 8086 microprocessor emulator (Labs 1–8)

emu8086 combines a source editor, an assembler and an emulator of the 8086 processor. It runs programs step by step and displays registers, flags, memory and the stack, and it emulates the DOS interrupt INT 21h used for keyboard input and screen output.

1. Install emu8086 (version 4.08) on Windows; it is installed on all lab computers.
2. Click **new**, choose **empty workspace**, and type the program.
3. Click **compile** to assemble the program and save the executable, or **emulate** to assemble and open the emulator directly.
4. In the emulator use **single step** to execute one instruction at a time and **run** to execute the whole program. Use **view → memory**, **view → stack** and **flags** to observe the machine state.

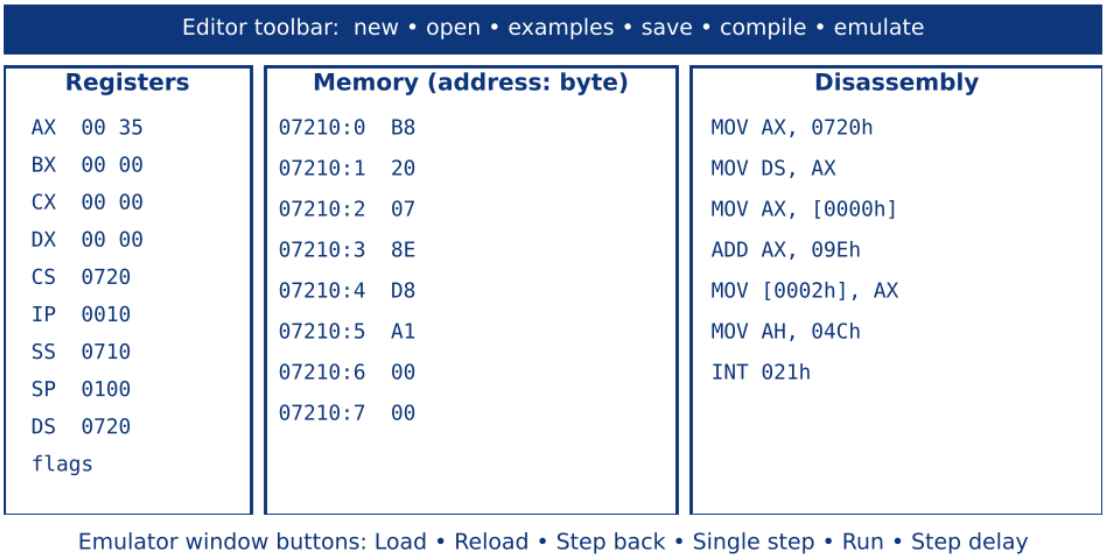


Figure 0.1: Layout of the emu8086 emulator window (schematic)

QtSPIM – MIPS32 simulator (Labs 9–13)

QtSPIM is a free, self-contained simulator that runs MIPS32 assembly programs. It loads a source file, assembles it, and shows the registers, the text segment (code) and the data segment while a separate console window shows program input/output.

1. Download QtSPIM from the official SPIM website (spimsimulator.sourceforge.net) for Windows, Linux or macOS and install it.
2. Write the program in any text editor (e.g. Notepad++) and save it with the extension `.asm` or `.s`.
3. In QtSPIM choose **File → Reinitialize and Load File** and select the program.
4. Press **Run/Continue (F5)** to run, or **Single Step (F10)** to execute one instruction at a time. Make sure **Simulator → Settings → Load exception handler** is ticked so that execution starts at the label `main`.

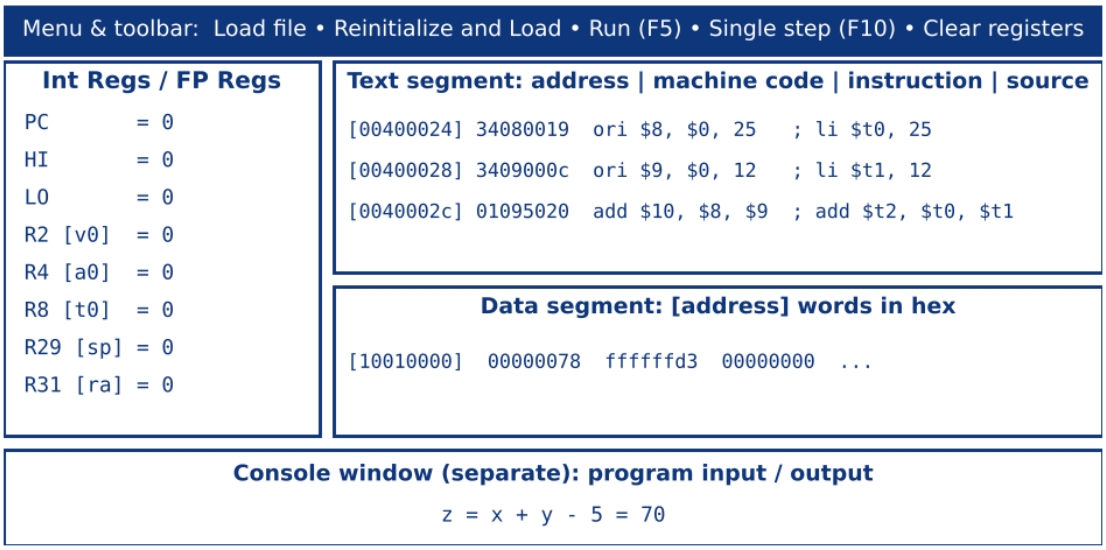


Figure 0.2: Layout of the QtSPIM window (schematic)

Note: all example programs in this workbook were tested before publication. Register values that depend on where the program is loaded (e.g. the value of DS in emu8086) may differ on your computer.

Lab Session 01

1. OBJECT

Exploring Instruction Set Architecture (ISA) of x86 Machines

Week / Lab Session	:	Week 1 / Lab 01
CLO Mapping	:	CLO1, CLO2
Time Required	:	3 hours
Software Required	:	emu8086 (8086 microprocessor emulator)

2. THEORY

2.1 Instruction Set Architecture (ISA)

The ISA of a machine is the set of its *attributes* that a system programmer needs to know to write system software, or that a compiler needs to translate a high-level language (HLL) program into machine language. Examples of such attributes are:

- *Instruction set* – the operations the processor can perform.
- *Programmer-accessible registers* – general-purpose registers, as opposed to special registers used only by the hardware and the operating system.
- *Memory–processor interaction* – how memory is addressed and how many bytes are read or written at a time.
- *Addressing modes* – ways of specifying operands (immediate, register, direct, indirect, indexed, ...).
- *Instruction formats* – the division of an instruction into fields (opcode, source and destination operands).

The ISA is also called the **programmer's view** or the **software model** of the machine.

2.2 The x86 family

The x86 ISA began in 1978 with the 16-bit Intel **8086** processor. Every later processor (80286, 80386, Pentium, Core i3/i5/i7) kept the 8086 instructions and added new ones, so a program written for the 8086 still runs on today's PCs. The 80386 introduced the 32-bit ISA (IA-32) with registers EAX, EBX, ...; modern processors use the 64-bit x86-64 ISA (RAX, RBX, ...). In this lab we use **emu8086**, which emulates the 16-bit 8086; the concepts carry over directly to IA-32 and x86-64.

2.3 Registers

Registers are storage locations inside the processor that can be accessed much faster than memory. The 8086 has fourteen 16-bit registers.

General-purpose (data) registers: AX, BX, CX and DX. Each can be used as a 16-bit register or as two independent 8-bit registers (high byte and low byte), as shown in Figure 1.1. In the 32-bit IA-32 ISA the same registers are extended to EAX, EBX, ECX and EDX.

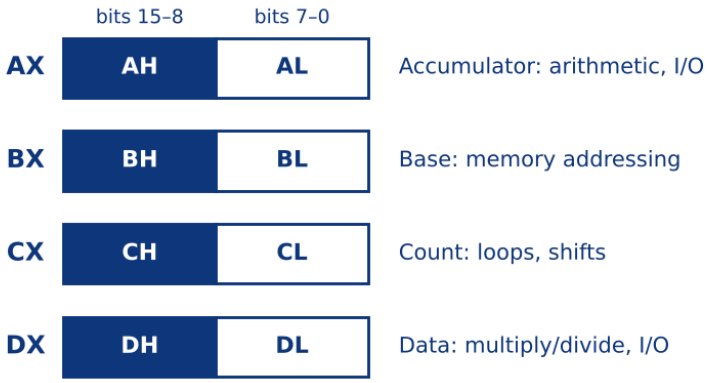


Figure 1.1: The general-purpose registers of the 8086 and their 8-bit halves

Group	Registers	Purpose
Index registers	SI, DI	Source and destination index; used for arrays and string operations.
Pointer registers	SP, BP, IP	SP = top of the stack, BP = base (reference) point in the stack, IP = offset of the next instruction to be executed.
Segment registers	CS, DS, SS, ES	Hold the starting addresses of the code, data, stack and extra segments.
Flags register	FLAGS	Individual bits report the result of an operation (status flags) or control the processor (control flags).

2.4 The FLAGS register

The FLAGS register is never used as a whole. Individual bits either **control** the operation of the CPU (control flags) or **reflect the outcome** of an arithmetic or logic operation (status flags).



Figure 1.2: Position of the flags in the 16-bit FLAGS register

Bit	Flag	Type	Meaning
11	OF (Overflow)	Status	1 when a signed result is too large or too small for the destination.
10	DF (Direction)	Control	Direction (forward/backward) for string instructions.
9	IF (Interrupt)	Control	1 = external (maskable) interrupts are enabled.
8	TF (Trap)	Control	1 = single-step mode (used by debuggers).
7	SF (Sign)	Status	Copy of the most significant bit of the result (1 = negative).
6	ZF (Zero)	Status	1 when the result is zero.
4	AF (Auxiliary carry)	Status	Carry out of bit 3; used for BCD arithmetic.
2	PF (Parity)	Status	1 when the low byte of the result has an even number of 1s.
0	CF (Carry)	Status	Carry out of (or borrow into) the most significant bit; unsigned overflow.

2.5 Memory addressing and segmentation

The 8086 has a 20-bit address bus, so it can address 2^20 bytes = **1 MB** of memory. Its registers, however, are only 16 bits wide. Memory is therefore divided into **segments** of up to 64 KB. A memory location is specified by a **segment** and an **offset** (written segment:offset), and the processor forms the 20-bit **physical address** as:

Physical address = Segment × 16 + Offset

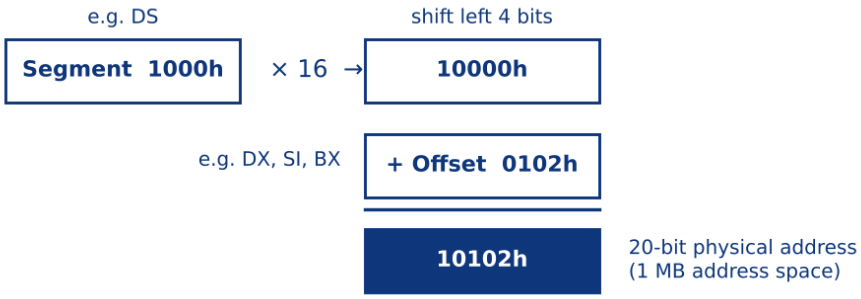


Figure 1.3: Forming a physical address from a segment and an offset

The code segment (CS) holds instructions and is used with IP; the data segment (DS) holds variables; the stack segment (SS) is used with SP and BP; the extra segment (ES) is an additional data segment. In 32-bit protected mode with a *flat memory model*, the operating system sets all segments to start at address 0, so programs simply use 32-bit offsets.

2.6 Byte ordering (endianness)

Memory is **byte-addressable**: every byte has its own address. A word (2 bytes) or double word (4 bytes) therefore occupies several consecutive addresses, and the order of the bytes can follow one of two conventions:

- **Little endian** – the least significant byte is stored at the lowest address. All Intel x86 processors are little endian.
- **Big endian** – the most significant byte is stored at the lowest address (e.g. MIPS in big-endian mode, IBM mainframes, network byte order).

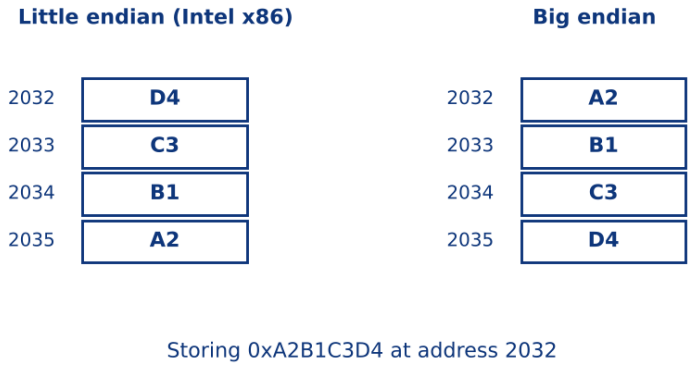


Figure 1.4: Storing 0xA2B1C3D4 at address 2032 in little- and big-endian machines

3. PROCEDURE

Program P1-1 does not print anything; its purpose is to let you **observe** registers, flags and memory in the emulator.

Program P1-1

```
; P1-1  Observing registers, flags and byte ordering
.model small
.stack 100h
.data
```

```
value dd 0A2B1C3D4h          ; 32-bit value stored in memory

.code
main proc
    mov ax, @data
    mov ds, ax

    mov ax, 1234h            ; AH = 12h, AL = 34h
    mov bl, ah               ; BL = 12h
    mov bh, al               ; BH = 34h  -> BX = 3412h

    mov al, 0FFh
    add al, 1                ; AL = 00h  -> CF = 1, ZF = 1

    mov al, 7Fh
    add al, 1                ; AL = 80h  -> OF = 1, SF = 1

    mov cx, word ptr value   ; low word  (stored first)  CX = C3D4h
    mov dx, word ptr value+2 ; high word (stored second) DX = A2B1h

    mov ah, 4Ch              ; return to operating system
    int 21h
main endp
end main
```

1. Open emu8086, click **new → empty workspace** and type program P1-1.
2. Click **emulate**. The emulator window opens with the first instruction highlighted.
3. Press **single step** repeatedly. After each step, record the registers and flags. Click the **flags** button to see the individual flags.
4. After `mov dx, word ptr value+2`, open **view → memory** and go to the address of `value` (DS:0000). Observe the order of the bytes D4, C3, B1, A2.

The expected values after each instruction are given below (the instructions that load DS are omitted).

Instruction executed	AX	BX	CX	DX	Flags after
mov ax, 1234h	1234	0000	0000	0000	CF=0 ZF=0 SF=0 OF=0
mov bl, ah	1234	0012	0000	0000	CF=0 ZF=0 SF=0 OF=0
mov bh, al	1234	3412	0000	0000	CF=0 ZF=0 SF=0 OF=0
mov al, 0FFh	12FF	3412	0000	0000	CF=0 ZF=0 SF=0 OF=0
add al, 1	1200	3412	0000	0000	CF=1 ZF=1 SF=0 OF=0
mov al, 7Fh	127F	3412	0000	0000	CF=1 ZF=1 SF=0 OF=0
add al, 1	1280	3412	0000	0000	CF=0 ZF=0 SF=1 OF=1
mov cx, value	1280	3412	C3D4	0000	CF=0 ZF=0 SF=1 OF=1
mov dx, word ptr value+2	1280	3412	C3D4	A2B1	CF=0 ZF=0 SF=1 OF=1

Observations: `mov bl, ah` and `mov bh, al` access the two halves of AX separately. `add al, 1` with AL = FFh gives 00h with a carry out (CF = 1, ZF = 1). `add al, 1` with AL = 7Fh gives 80h: two positive signed numbers produced a negative result, so OF = 1 and SF = 1. The low word of `value` (C3D4h) is stored first, confirming that the 8086 is little endian.

4. EXERCISES

a) Fill in the following tables to show the storage of **0xABDADDBA** at address 1996 in the memory of a machine using (i) little-endian and (ii) big-endian byte ordering.

Address	Little endian	Big endian
1996		
1997		
1998		
1999		

b) What is the significance of learning the ISA of a processor? Give two examples.

c) Draw the CX register showing the size and position of CH, CL and CX within it. What will CX contain after `mov ch, 0Ah` followed by `mov cl, 0F3h`?

d) Compute the physical address for (i) DS = 2000h, offset = 0150h (ii) CS = 1A2Bh, IP = 0F0Fh. Can two different segment:offset pairs give the same physical address? Give an example.

e) For each instruction below, assume AX contains the given value before execution. Use emu8086 to find AX after execution and the values of CF, OF, SF, PF, AF and ZF. All numbers are in hex.

AX before	Instruction	AX after	CF	OF	SF	PF	AF	ZF
0045	add ax, 45h							
FF45	add ax, 45h							
0045	add ax, -45h							
FF45	add ax, -45h							
FFFF	add ax, 1							
7FFF	add ax, 1							

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab Session 02

1. OBJECT

Learning to program in Assembly Language of x86 Machines

Week / Lab Session : Week 2 / Lab 02
CLO Mapping : CLO1, CLO2
Time Required : 3 hours
Software Required : emu8086 (8086 microprocessor emulator)

2. THEORY

We present a short but complete program (P2-1) to explain the basics of assembly language programming in emu8086. It adds 158 to a number stored in memory and stores the result in another memory variable.

Program P2-1

```
; P2-1  Example Assembly Program
; Adds 158 to a number stored in memory
.model small
.stack 100h

.data
number dw -105
sum    dw ?

.code
main proc
    mov ax, @data       ; initialise DS with the data segment address
    mov ds, ax

    mov ax, number      ; AX = -105
    add ax, 158         ; AX = -105 + 158 = 53
    mov sum, ax         ; store the result in memory

    mov ah, 4Ch         ; terminate the program
    int 21h
main endp
end main
```

2.1 Program explanation

Comments begin with a semicolon (;) and continue to the end of the line. They are essential because assembly programs are difficult to understand without explanation.

Directives instruct the assembler to perform specific actions; unlike instructions they do not generate machine code.

Directive	Purpose (tells the assembler)
.model small	Use the small memory model: one code segment and one data segment (each up to 64 KB).
.stack 100h	Reserve 100h (256) bytes for the stack.
.data	Beginning of the data segment (variables).
.code	Beginning of the code segment (instructions).

Directive	Purpose (tells the assembler)
main proc / main endp	Beginning and end of the procedure named main.
end main	End of the program; execution starts at main.
db, dw, dd	Define byte (8 bits), word (16 bits), double word (32 bits).
dup	Duplicate a value, e.g. `buffer db 5 dup(0)` reserves 5 bytes of 0.
equ	Define a named constant, e.g. `len equ 8` (no memory is allocated).

Instructions: `mov ax, @data` and `mov ds, ax` load the address of the data segment into DS (a segment register cannot be loaded with an immediate value directly). `mov ax, number` copies the word at memory variable `number` into AX, `add ax, 158` adds 158 to AX, and `mov sum, ax` copies AX into the variable `sum`. Finally, **INT 21h with AH = 4Ch** returns control to the operating system.

Identifiers (names of variables, labels and procedures) consist of letters, digits and the underscore, cannot begin with a digit, and cannot be a reserved word (mnemonics, register names, directives). The assembler is not case-sensitive, but it is good practice to write instructions in lowercase.

2.2 Defining data and displaying a message

Program P2-2 shows the main data definitions and uses DOS function **09h** to display a string. The string must end with the character `$`, and DX must contain its offset.

Program P2-2

```
; P2-2  Data definition directives and displaying a message
.model small
.stack 100h

.data
msg     db 'Welcome to Computer Architecture Lab!$'
byteVar db 25            ; 8-bit variable
wordVar dw 1000          ; 16-bit variable
list    db 1, 2, 3, 4    ; array of 4 bytes
buffer  db 5 dup(0)      ; 5 bytes initialised to 0

.code
main proc
    mov ax, @data
    mov ds, ax

    lea dx, msg          ; DX = offset of the message
    mov ah, 09h          ; DOS function 09h: display string ending in '$'
    int 21h

    mov al, byteVar      ; AL = 25 (19h)
    mov bx, wordVar      ; BX = 1000 (03E8h)
    mov cl, list+2       ; CL = third element = 3

    mov ah, 4Ch
    int 21h
main endp
end main
```

Output

```
Welcome to Computer Architecture Lab!
```

2.3 Assembling and the listing file

When a program is compiled, emu8086 translates each instruction into machine code and produces a **listing** that shows, for every line, its **offset** in the segment, the generated **machine code (opcode and operands in hex)** and the source statement. In the emulator the same information appears in the disassembly and memory windows. For example, `add ax, 158` assembles to `05 9E 00`: opcode 05h (ADD AX, imm16) followed by the immediate 009Eh stored in little-endian order.

3. PROCEDURE

1. Open the emu8086 emulator, create a new file (**empty workspace**) and type program P2-1.
2. Click **compile**, save the file as `lab2_p1.asm`, and check that the assembler reports no errors.
3. Click **emulate** to load the program into the emulator.
4. Use **single step** to execute the program one instruction at a time.
5. Observe the register window after every step and the **memory** window (DS:0000) to see `number` and `sum`.
6. Observe the flags after `add ax, 158`.
7. Record the effect of each instruction:

Instruction executed	AX	BX	CX	DX	Flags after
mov ax, number	FF97	0000	0000	0000	CF=0 ZF=0 SF=0 OF=0
add ax, 158	0035	0000	0000	0000	CF=1 ZF=0 SF=0 OF=0
mov sum, ax	0035	0000	0000	0000	CF=1 ZF=0 SF=0 OF=0

After `mov ax, number`, AX = FF97h, which is −105 in two's complement. Adding 158 gives 0035h = 53. The addition produces a carry out of bit 15, so CF = 1, but the signed result is correct, so OF = 0.

4. EXERCISES

a) Examine the flags after `add ax, 158` in P2-1 and explain each value.

Flag	Value (0/1)	Reason for this value
CF		
OF		
ZF		
SF		

b) Which instruction changes memory in P2-1? What change do you observe in the memory window? Do the flags change as a result of this instruction? Why?

c) Fill in the offsets of the instructions in the code segment of P2-1 using the emulator or the listing.

Offset	Instruction	Machine code (hex)
	mov ax, number	
	add ax, 158	
	mov sum, ax	
	mov ah, 4Ch	
	int 21h	

d) Modify P2-1 to change the value of `number` to −250 and to add 74 instead of 158. Assemble and execute the program. Explain the changes in the registers, flags and memory after each instruction. Submit a screenshot.

e) Modify P2-1 to add two numbers stored in memory as `number1` and `number2` (Hint: copy `number1` to AX and then use an appropriate ADD instruction). Store the total in `sum`. Submit the program and a screenshot.

f) Write a program that displays your name on the first line and your registration number on the second line (use 13, 10 for a new line inside the string, e.g. `msg db 'Ali', 13, 10, '$'`).

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab Session 03

1. OBJECT

Using MACROS for Input / Output and Data Conversion

Week / Lab Session : Week 3 / Lab 03
CLO Mapping : CLO1, CLO2
Time Required : 3 hours
Software Required : emu8086 (8086 microprocessor emulator)

2. THEORY

2.1 Input and output through DOS (INT 21h)

Assembly language has no input/output statements. Programs request services from the operating system through **software interrupts**. DOS services are called by placing a function number in AH and executing `int 21h`. emu8086 emulates these services.

AH	Service	Input	Output
01h	Read a character from the keyboard (with echo)	–	AL = ASCII code of the key
02h	Display one character	DL = ASCII code	character on the screen
09h	Display a string ending in '$'	DS:DX = address of string	string on the screen
0Ah	Read a string (buffered input)	DS:DX = address of buffer	buffer[1] = length, buffer[2..] = characters
4Ch	Terminate the program	AL = return code	control returns to the OS

2.2 Macros

A **macro** is a named block of statements. Wherever the macro name is used, the assembler replaces it with the block (this is called **macro expansion**). Macros make programs shorter and easier to read. Parameters are written after the word MACRO and are replaced by the arguments given at the call.

Syntax

```
name MACRO param1, param2
    ; statements that use param1 and param2
ENDM
```

Our I/O macros save and restore the registers they use (PUSH/POP), so calling them does not disturb the values held in AX and DX by the main program.

2.3 Data conversion

The keyboard and screen work with **ASCII codes**, while arithmetic instructions work with **binary numbers**. Conversion is therefore needed:

- **ASCII digit → number:** subtract 30h ('0' = 30h, '7' = 37h, so 37h − 30h = 7).
- **Number (0–9) → ASCII digit:** add 30h.
- **Lowercase ↔ uppercase:** the codes differ by 20h ('a' = 61h, 'A' = 41h).
- **Binary number → decimal string:** divide repeatedly by 10; each remainder is a digit (least significant first). Push the digits on the stack and pop them to print them in the correct order.

- **Binary number → hexadecimal:** take 4 bits (a nibble) at a time; values 0–9 map to '0'–'9' and 10–15 to 'A'–'F' (add 37h).

3. PROCEDURE

Type, assemble and run each program. Enter the inputs shown and compare your output with the expected output.

Program P3-1

```
; P3-1  Macros for input/output: read a lowercase letter, display it in uppercase
.model small
.stack 100h

; ---------- macro definitions ----------
print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

readch macro             ; read one key with echo, result in AL
    mov ah, 01h
    int 21h
endm

putch macro char           ; display one character
    mov dl, char
    mov ah, 02h
    int 21h
endm

.data
prompt db 'Enter a lowercase letter: $'
result db 'In uppercase: $'

.code
main proc
    mov ax, @data
    mov ds, ax

    print prompt
    readch               ; AL = ASCII code of the key, e.g. 'g' = 67h
    mov bl, al
```

```
    sub bl, 20h          ; 'a'(61h) - 20h = 'A'(41h)
    newline
    print result
    putch bl

    mov ah, 4Ch
    int 21h
main endp
end main
```

Output (input: g)

```
Enter a lowercase letter: g
In uppercase: G
```

Program P3-2

```
; P3-2  Data conversion: ASCII digits -> number -> decimal display
.model small
.stack 100h

print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

.data
msg1 db 'Enter first digit : $'
msg2 db 'Enter second digit: $'
msg3 db 'Sum = $'
num1 db ?

.code
main proc
    mov ax, @data
    mov ds, ax

    print msg1
    mov ah, 01h          ; read first digit, e.g. '7' = 37h
    int 21h
    sub al, 30h          ; ASCII -> number : 37h - 30h = 7
```

```
    mov num1, al
    newline

    print msg2
    mov ah, 01h          ; read second digit
    int 21h
    sub al, 30h
    add al, num1         ; AL = binary sum (0..18)
    newline

    mov ah, 0            ; AX = sum
    push ax
    print msg3
    pop ax
    call print_num       ; number -> decimal ASCII on screen

    mov ah, 4Ch
    int 21h
main endp

; print_num: display the unsigned number in AX in decimal
print_num proc
    push ax
    push bx
    push cx
    push dx
    mov cx, 0            ; digit counter
    mov bx, 10
pn_div:
    mov dx, 0
    div bx               ; AX = AX / 10, DX = remainder (next digit)
    push dx              ; save digit on stack
    inc cx
    cmp ax, 0
    jne pn_div
pn_out:
    pop dx               ; digits come out in the correct order
    add dl, 30h          ; number -> ASCII
    mov ah, 02h
    int 21h
    loop pn_out
    pop dx
    pop cx
    pop bx
    pop ax
    ret
print_num endp
end main
```

Output (inputs: 7 and 9)

```
Enter first digit : 7
Enter second digit: 9
Sum = 16
```

`print_num` is a **procedure** (called with CALL and ended with RET). Procedures are studied in detail in Lab 08; here it is used as a ready-made routine to convert a binary number into decimal digits.

Program P3-3

```
; P3-3  Buffered string input (INT 21h, AH = 0Ah) and hexadecimal display
.model small
.stack 100h

print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

.data
prompt db 'Enter your name: $'
hello  db 'Hello, $'
lenmsg db 'Length of name (hex) = $'
buf    db 20             ; byte 0: maximum characters allowed
       db ?              ; byte 1: characters actually typed (filled by DOS)
       db 20 dup('$')    ; bytes 2..: the characters themselves

.code
main proc
    mov ax, @data
    mov ds, ax

    print prompt
    lea dx, buf
    mov ah, 0Ah          ; buffered keyboard input
    int 21h
    newline

    print hello
    lea dx, buf+2        ; the typed text starts at buf+2
    mov ah, 09h
    int 21h
    newline

    print lenmsg
    mov al, buf+1        ; number of characters typed
    mov bl, al
    mov cl, 4
    shr al, cl           ; high nibble
    call hex_digit
```

```
    mov al, bl
    and al, 0Fh          ; low nibble
    call hex_digit

    mov ah, 4Ch
    int 21h
main endp

; hex_digit: display the value 0..15 in AL as one hex character
hex_digit proc
    cmp al, 9
    jbe hd_num
    add al, 7            ; 'A'..'F' are 7 places after '9'+1
hd_num:
    add al, 30h
    mov dl, al
    mov ah, 02h
    int 21h
    ret
hex_digit endp
end main
```

Output (input: Ayesha Khan)

```
Enter your name: Ayesha Khan
Hello, Ayesha Khan
Length of name (hex) = 0B
```

4. EXERCISES

a) Single-step P3-1 and write the value of AL after `readch` and BL after `sub bl, 20h` for the input 'g'. What happens if you enter an uppercase letter? Modify the program so that it converts **uppercase to lowercase** as well.

b) Look at the code generated where the macro `print` is used (view the disassembly in the emulator). How many instructions does each use of the macro produce? What is the difference between using a macro 10 times and writing the same code once in a procedure?

c) Modify P3-2 to read two digits and display their **difference** and **product** (e.g. 9 − 4 = 5, 9 × 4 = 36).

d) Write a macro `read_num` that reads a two-digit decimal number (e.g. 47) from the keyboard and leaves its binary value in AL. (Hint: value = first digit × 10 + second digit.)

e) Write a program that reads a character and displays its ASCII code in **hexadecimal** and in **binary** (8 bits). Use the `hex_digit` procedure of P3-3 and a loop with `shl al, 1` for the binary display.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab Session 04

1. OBJECT

Using x86 Data Transfer and Arithmetic Instructions

Week / Lab Session : Week 4 / Lab 04
CLO Mapping : CLO1, CLO2
Time Required : 3 hours
Software Required : emu8086 (8086 microprocessor emulator)

2. THEORY

2.1 Data transfer instructions

Instruction	Operation	Notes
MOV dest, src	dest ← src	Both operands must be the same size; memory-to-memory and immediate-to-segment moves are not allowed.
XCHG op1, op2	op1 ↔ op2	Swaps two registers or a register and memory.
LEA reg, mem	reg ← offset of mem	Loads the address, not the contents.
PUSH src / POP dest	stack ← src / dest ← stack	Word operands only; the stack is last-in, first-out (LIFO).
XLAT	AL ← [BX + AL]	Table look-up: BX = table address, AL = index.

2.2 Addressing modes

Mode	Example	Operand is ...
Immediate	mov cx, 0ABCDh	a constant in the instruction
Register	mov bx, ax	in a register
Direct	mov ax, var1	in memory at the address of var1
Register indirect	mov ax, [bx]	in memory at the address in BX (or SI, DI, BP)
Based / indexed	mov al, list[si]	in memory at list + SI

2.3 Arithmetic instructions

Instruction	Operation	Flags affected
ADD / ADC	dest ← dest + src (+ CF)	CF, ZF, SF, OF, PF, AF
SUB / SBB	dest ← dest − src (− CF)	CF, ZF, SF, OF, PF, AF
INC / DEC	dest ← dest ± 1	all except CF
NEG	dest ← 0 − dest (two's complement)	CF, ZF, SF, OF, PF, AF
MUL src (unsigned) / IMUL src (signed)	8-bit: AX ← AL × src; 16-bit: DX:AX ← AX × src	CF = OF = 1 if the upper half is used
DIV src (unsigned) / IDIV src (signed)	8-bit: AL ← AX / src, AH ← remainder; 16-bit: AX ← DX:AX / src, DX ← remainder	undefined; divide error if the quotient does not fit

Important: before a 16-bit unsigned DIV, DX must be cleared (`mov dx, 0`), because the dividend is DX:AX. For a signed IDIV use `cwd` to extend the sign of AX into DX.

3. PROCEDURE

Program P4-1

```
; P4-1  Data transfer instructions: MOV, XCHG, LEA, PUSH/POP, XLAT
.model small
.stack 100h

.data
var1   dw 1111h
var2   dw 2222h
table  db '0123456789ABCDEF'   ; look-up table for XLAT

.code
main proc
    mov ax, @data
    mov ds, ax

    mov ax, var1        ; memory  -> register : AX = 1111h
    mov bx, ax          ; register-> register : BX = 1111h
    mov cx, 0ABCDh      ; immediate-> register: CX = ABCDh
    mov var2, cx        ; register-> memory   : var2 = ABCDh

    xchg ax, cx         ; swap        : AX = ABCDh, CX = 1111h
    xchg ah, al         ; swap bytes  : AX = CDABh

    push ax             ; save AX on the stack
    push bx             ; save BX on the stack
    pop  dx             ; DX = 1111h  (last in, first out)
    pop  si             ; SI = CDABh

    lea bx, table       ; BX = address of the table
    mov al, 0Ch         ; index 12
    xlat                ; AL = table[12] = 'C' (43h)

    mov ah, 4Ch
    int 21h
main endp
end main
```

1. Type P4-1, click **emulate** and execute it with **single step**.
2. Open **view → stack** before the PUSH instructions and watch SP decrease by 2 for every PUSH and increase by 2 for every POP.
3. Compare your register values with the table below.

Instruction executed	AX	BX	CX	DX	Flags after
mov ax, var1	1111	0000	0000	0000	CF=0 ZF=0 SF=0 OF=0
mov bx, ax	1111	1111	0000	0000	CF=0 ZF=0 SF=0 OF=0
mov cx, 0ABCDh	1111	1111	ABCD	0000	CF=0 ZF=0 SF=0 OF=0
mov var2, cx	1111	1111	ABCD	0000	CF=0 ZF=0 SF=0 OF=0
xchg ax, cx	ABCD	1111	1111	0000	CF=0 ZF=0 SF=0 OF=0
xchg ah, al	CDAB	1111	1111	0000	CF=0 ZF=0 SF=0 OF=0
push ax	CDAB	1111	1111	0000	CF=0 ZF=0 SF=0 OF=0
push bx	CDAB	1111	1111	0000	CF=0 ZF=0 SF=0 OF=0
pop dx	CDAB	1111	1111	1111	CF=0 ZF=0 SF=0 OF=0

Instruction executed	AX	BX	CX	DX	Flags after
pop si	CDAB	1111	1111	1111	CF=0 ZF=0 SF=0 OF=0
lea bx, table	CDAB	0106	1111	1111	CF=0 ZF=0 SF=0 OF=0
mov al, 0Ch	CD0C	0106	1111	1111	CF=0 ZF=0 SF=0 OF=0
xlat	CD43	0106	1111	1111	CF=0 ZF=0 SF=0 OF=0

Note that the value of BX after `lea bx, table` depends on where the table is placed in the data segment, so it may differ in your emulator.

Program P4-2

```
; P4-2  Arithmetic instructions: evaluate  result = (A * B + C) / D
.model small
.stack 100h

print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

.data
numA dw 25
numB dw 12
numC dw 7
numD dw 9
m1   db 'A * B         = $'
m2   db 'A * B + C     = $'
m3   db '(A*B + C) / D = $'
m4   db 'remainder     = $'
m5   db 'C - A         = $'

.code
main proc
    mov ax, @data
    mov ds, ax

    mov ax, numA
    mul numB               ; DX:AX = A * B = 300 (unsigned multiply)
    print m1
    call print_snum
```

```
    newline

    add ax, numC            ; AX = 307
    print m2
    call print_snum
    newline

    mov dx, 0           ; clear DX before a 16-bit divide
    div numD               ; AX = 307 / 9 = 34, DX = 307 mod 9 = 1
    mov bx, dx          ; keep the remainder
    print m3
    call print_snum
    newline
    print m4
    mov ax, bx
    call print_snum
    newline

    mov ax, numC
    sub ax, numA            ; AX = C - A = 7 - 25 = -18 (negative result, SF = 1)
    print m5
    call print_snum

    mov ah, 4Ch
    int 21h
main endp

; print_snum: display the signed number in AX in decimal
print_snum proc
    push ax
    push bx
    push cx
    push dx
    test ax, ax
    jns ps_pos
    push ax
    mov dl, '-'          ; negative: print the sign
    mov ah, 02h
    int 21h
    pop ax
    neg ax               ; make it positive (two's complement)
ps_pos:
    mov cx, 0
    mov bx, 10
ps_div:
    mov dx, 0
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne ps_div
ps_out:
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    loop ps_out
    pop dx
    pop cx
```

```
    pop bx
    pop ax
    ret
print_snum endp
end main
```

Output

```
A * B         = 300
A * B + C     = 307
(A*B + C) / D = 34
remainder     = 1
C - A         = -18
```

4. EXERCISES

a) In P4-1, which instructions change the flags? Which instruction is used to implement a table look-up? Explain the value of SI after the two POP instructions.

b) Using MUL, evaluate `result = A² + 2AB + B²` for A = 15, B = 9 and display the result. Verify that it equals $(A + B)^2$.

c) Modify P4-2 to use **IMUL** and **IDIV** and evaluate (A × B + C) / D for A = −25, B = 12, C = 7, D = 9. Explain why `cwd` is needed before IDIV.

d) Write a program that converts a temperature in Celsius (stored in a variable) to Fahrenheit using F = C × 9 / 5 + 32 and displays both values. Test with C = 37 and C = −40.

e) Predict, then verify in emu8086, the contents of the destination and the flags after each instruction.

Initial values	Instruction	Result	CF	ZF	SF	OF
AL = 0F0h, BL = 20h	add al, bl					
AL = 05h, BL = 07h	sub al, bl					
AX = 7FFFh	inc ax					
AL = 80h	neg al					
AL = 30h, BL = 06h	mul bl					
AX = 0064h, BL = 07h	div bl					

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab Session 05

1. OBJECT

Implementing Branching in x86 Assembly Language

Week / Lab Session : Week 5 / Lab 05
CLO Mapping : CLO1, CLO2
Time Required : 3 hours
Software Required : emu8086 (8086 microprocessor emulator)

2. THEORY

2.1 Comparing and jumping

High-level decisions (`if`, `if-else`, `switch`) are implemented in assembly with a **comparison** that sets the flags, followed by a **conditional jump** that tests the flags. `CMP dest, src` computes dest − src, sets the flags and discards the result. `TEST dest, src` performs a bitwise AND and sets the flags. `JMP label` transfers control unconditionally.

Condition	Unsigned jump	Signed jump	Flags tested
equal / zero	JE (JZ)	JE (JZ)	ZF = 1
not equal	JNE (JNZ)	JNE (JNZ)	ZF = 0
greater	JA (JNBE)	JG (JNLE)	unsigned: CF = 0 and ZF = 0; signed: ZF = 0 and SF = OF
greater or equal	JAE (JNB)	JGE (JNL)	unsigned: CF = 0; signed: SF = OF
less	JB (JNAE)	JL (JNGE)	unsigned: CF = 1; signed: SF ≠ OF
less or equal	JBE (JNA)	JLE (JNG)	unsigned: CF = 1 or ZF = 1; signed: ZF = 1 or SF ≠ OF
single flags	JC / JNC, JO / JNO, JS / JNS, JP / JNP		CF, OF, SF, PF

Use **above/below** (JA, JB) for unsigned numbers and **greater/less** (JG, JL) for signed numbers. The same bit pattern can be large when unsigned but negative when signed: 0FFh = 255 unsigned but −1 signed (program P5-3).

2.2 Translating if-else

Pattern

```
; if (x >= y) then  max = x  else  max = y
    mov ax, x
    cmp ax, y
    jge then_part        ; condition true -> then part
    mov ax, y            ; else part
    jmp end_if
then_part:
    ; ax already holds x
end_if:
    mov max, ax
```

3. PROCEDURE

The following programs use the `print` and `newline` macros of Lab 03 and the procedure `print_snum`, which displays the signed number in AX.

Program P5-1

```
; P5-1  Branching: even or odd, and the larger of two signed numbers
.model small
.stack 100h

print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

.data
ask   db 'Enter a digit (0-9): $'
evenm db ' is EVEN$'
oddm  db ' is ODD$'
x     dw -12
y     dw 7
bigm  db 'Larger of X and Y = $'

.code
main proc
    mov ax, @data
    mov ds, ax

    ; ---- if (digit is even) ... else ... ----
    print ask
    mov ah, 01h
    int 21h              ; AL = ASCII digit
    sub al, 30h
    test al, 1           ; ZF = 1 when bit 0 is 0 (even)
    jnz is_odd
    print evenm
    jmp done_eo
is_odd:
    print oddm
done_eo:
    newline
```

```
    ; ---- signed comparison: max(X, Y) ----
    mov ax, x
    cmp ax, y            ; compares -12 with 7
    jge x_bigger         ; signed jump: -12 >= 7 is false
    mov ax, y
x_bigger:
    print bigm
    call print_snum
    newline

    mov ah, 4Ch
    int 21h
main endp

; print_snum: display the signed number in AX in decimal
print_snum proc
    push ax
    push bx
    push cx
    push dx
    test ax, ax
    jns ps_pos
    push ax
    mov dl, '-'          ; negative: print the sign
    mov ah, 02h
    int 21h
    pop ax
    neg ax               ; make it positive (two's complement)
ps_pos:
    mov cx, 0
    mov bx, 10
ps_div:
    mov dx, 0
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne ps_div
ps_out:
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    loop ps_out
    pop dx
    pop cx
    pop bx
    pop ax
    ret
print_snum endp
end main
```

Output (input: 7)

```
Enter a digit (0-9): 7 is ODD
Larger of X and Y = 7
```

Program P5-2

```
; P5-2  Multi-way branching (if / else-if / else): grade from marks
.model small
.stack 100h

print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

.data
marks  dw 74
mmsg   db 'Marks = $'
gmsg   db '   Grade = $'

.code
main proc
    mov ax, @data
    mov ds, ax

    mov ax, marks
    print mmsg
    call print_snum
    print gmsg

    cmp ax, 85           ; if (marks >= 85) grade = 'A'
    jae grade_a
    cmp ax, 70           ; else if (marks >= 70) grade = 'B'
    jae grade_b
    cmp ax, 55           ; else if (marks >= 55) grade = 'C'
    jae grade_c
    cmp ax, 50           ; else if (marks >= 50) grade = 'D'
    jae grade_d
    mov dl, 'F'          ; else grade = 'F'
    jmp show
grade_a:
    mov dl, 'A'
    jmp show
grade_b:
    mov dl, 'B'
    jmp show
```

```
grade_c:
    mov dl, 'C'
    jmp show
grade_d:
    mov dl, 'D'
show:
    mov ah, 02h
    int 21h

    mov ah, 4Ch
    int 21h
main endp

; print_snum: display the signed number in AX in decimal
print_snum proc
    push ax
    push bx
    push cx
    push dx
    test ax, ax
    jns ps_pos
    push ax
    mov dl, '-'          ; negative: print the sign
    mov ah, 02h
    int 21h
    pop ax
    neg ax               ; make it positive (two's complement)
ps_pos:
    mov cx, 0
    mov bx, 10
ps_div:
    mov dx, 0
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne ps_div
ps_out:
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    loop ps_out
    pop dx
    pop cx
    pop bx
    pop ax
    ret
print_snum endp
end main
```

Output

```
Marks = 74   Grade = B
```

Program P5-3

```
; P5-3  Signed vs unsigned comparison of the same bit patterns
.model small
```

```
.stack 100h

print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

.data
m1 db 'Unsigned: 0FFh > 01h ? $'
m2 db 'Signed  : 0FFh > 01h ? $'
yes db 'YES$'
no  db 'NO$'

.code
main proc
    mov ax, @data
    mov ds, ax

    mov al, 0FFh         ; 255 unsigned, -1 signed
    mov bl, 01h

    print m1
    cmp al, bl
    ja  u_yes            ; JA uses CF and ZF (unsigned)
    print no
    jmp u_done
u_yes:
    print yes
u_done:
    newline

    print m2
    cmp al, bl
    jg  s_yes            ; JG uses SF, OF and ZF (signed)
    print no
    jmp s_done
s_yes:
    print yes
s_done:

    mov ah, 4Ch
```

```
    int 21h
main endp
end main
```

Output

```
Unsigned: 0FFh > 01h ? YES
Signed  : 0FFh > 01h ? NO
```

4. EXERCISES

a) Single-step P5-3 and record CF, ZF, SF and OF after `cmp al, bl`. Use these values to explain why JA jumps but JG does not.

b) Write a program that reads a character and displays whether it is an **uppercase letter**, a **lowercase letter**, a **digit** or **another symbol**.

c) Write a program that finds and displays the **largest of three** signed numbers stored in memory.

d) Modify P5-2 so that the marks are read from the keyboard (two digits) and the program displays an error message if the marks are greater than 100.

e) Write a program that reads a year (stored in a variable) and displays whether it is a **leap year** (divisible by 4 and not by 100, or divisible by 400). Test with 2024, 1900 and 2000.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab Session 06

1. OBJECT

Implementation of Loop Structures in x86 Assembly Language

Week / Lab Session : Week 6 / Lab 06
CLO Mapping : CLO1, CLO2
Time Required : 3 hours
Software Required : emu8086 (8086 microprocessor emulator)

2. THEORY

2.1 The LOOP instruction

`LOOP label` decrements CX and jumps to the label if CX is not zero. It implements a **counter-controlled** (for) loop: load the repetition count into CX before the loop. Related instructions are `LOOPE/LOOPZ` (loop while CX ≠ 0 and ZF = 1), `LOOPNE/LOOPNZ` (loop while CX ≠ 0 and ZF = 0) and `JCXZ` (jump if CX = 0), which protects against starting a loop with CX = 0 (it would run 65,536 times).

2.2 Loop structures in assembly

High-level structure	Assembly pattern
for i = N downto 1	mov cx, N / top: ... / loop top
while (condition)	top: compare / jump to end if condition false / body / jmp top / end:
do ... while (condition)	top: body / compare / jump to top if condition true
sentinel-controlled	read a value / compare with the sentinel (e.g. Enter = 13) / exit or process and repeat
nested loops	save the outer counter (PUSH CX, or use another register) before the inner loop

3. PROCEDURE

Program P6-1

```
; P6-1  Counted loops with LOOP: print 1..9 and the sum 1+2+...+N
.model small
.stack 100h

print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
```

```
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

.data
nums db 'Counting  : $'
summ db 'Sum 1..N  = $'
n    dw 10

.code
main proc
    mov ax, @data
    mov ds, ax

    ; ---- for i = 1 to 9 : print i ----
    print nums
    mov cx, 9            ; loop counter
    mov dl, '1'
next_digit:
    mov ah, 02h
    int 21h              ; display DL
    inc dl
    loop next_digit      ; CX = CX - 1, jump if CX != 0
    newline

    ; ---- sum = 1 + 2 + ... + N ----
    mov cx, n            ; CX = N = 10
    mov ax, 0
add_next:
    add ax, cx           ; adds N, N-1, ..., 1
    loop add_next
    print summ
    call print_snum      ; 55
    newline

    mov ah, 4Ch
    int 21h
main endp

; print_snum: display the signed number in AX in decimal
print_snum proc
    push ax
    push bx
    push cx
    push dx
    test ax, ax
    jns ps_pos
    push ax
    mov dl, '-'          ; negative: print the sign
    mov ah, 02h
    int 21h
    pop ax
    neg ax               ; make it positive (two's complement)
ps_pos:
    mov cx, 0
    mov bx, 10
```

```
ps_div:
    mov dx, 0
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne ps_div
ps_out:
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    loop ps_out
    pop dx
    pop cx
    pop bx
    pop ax
    ret
print_snum endp
end main
```

Output

```
Counting  : 123456789
Sum 1..N  = 55
```

Program P6-2

```
; P6-2  Nested loops (triangle of stars) and a while loop (factorial)
.model small
.stack 100h

print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

.data
rows  dw 5
facm  db '6! = $'
```

```
.code
main proc
    mov ax, @data
    mov ds, ax

    ; ---- nested loop: row r prints r stars ----
    mov bx, 1            ; BX = current row
outer:
    mov cx, bx           ; inner loop runs BX times
inner:
    mov dl, '*'
    mov ah, 02h
    int 21h
    loop inner
    newline
    inc bx
    cmp bx, rows
    jbe outer            ; repeat while row <= 5

    ; ---- while (n > 1) { fact = fact * n; n-- } ----
    mov ax, 1            ; fact
    mov cx, 6            ; n
while_top:
    cmp cx, 1
    jbe while_end        ; exit when n <= 1
    mul cx               ; DX:AX = AX * CX
    dec cx
    jmp while_top
while_end:
    print facm
    call print_snum      ; 720
    newline

    mov ah, 4Ch
    int 21h
main endp

; print_snum: display the signed number in AX in decimal
print_snum proc
    push ax
    push bx
    push cx
    push dx
    test ax, ax
    jns ps_pos
    push ax
    mov dl, '-'          ; negative: print the sign
    mov ah, 02h
    int 21h
    pop ax
    neg ax               ; make it positive (two's complement)
ps_pos:
    mov cx, 0
    mov bx, 10
ps_div:
    mov dx, 0
    div bx
    push dx
    inc cx
```

```
    cmp ax, 0
    jne ps_div
ps_out:
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    loop ps_out
    pop dx
    pop cx
    pop bx
    pop ax
    ret
print_snum endp
end main
```

Output

```
*
**
***
****
*****
6! = 720
```

In P6-2 the inner loop uses CX while the row number is kept in BX, so the two loops do not interfere. The factorial loop is a *while* loop: the condition is tested at the top before each iteration.

Program P6-3

```
; P6-3  Sentinel-controlled loop: count the vowels in a line typed by the user
.model small
.stack 100h

print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

.data
ask  db 'Type a sentence and press Enter: $'
```

```
res  db 'Number of vowels = $'

.code
main proc
    mov ax, @data
    mov ds, ax

    print ask
    mov bx, 0            ; vowel counter
read_loop:
    mov ah, 01h
    int 21h              ; AL = next character
    cmp al, 13           ; Enter key ends the loop
    je  read_end
    or  al, 20h          ; convert letters to lowercase
    cmp al, 'a'
    je  vowel
    cmp al, 'e'
    je  vowel
    cmp al, 'i'
    je  vowel
    cmp al, 'o'
    je  vowel
    cmp al, 'u'
    je  vowel
    jmp read_loop
vowel:
    inc bx
    jmp read_loop
read_end:
    newline
    print res
    mov ax, bx
    call print_snum

    mov ah, 4Ch
    int 21h
main endp

; print_snum: display the signed number in AX in decimal
print_snum proc
    push ax
    push bx
    push cx
    push dx
    test ax, ax
    jns ps_pos
    push ax
    mov dl, '-'          ; negative: print the sign
    mov ah, 02h
    int 21h
    pop ax
    neg ax               ; make it positive (two's complement)
ps_pos:
    mov cx, 0
    mov bx, 10
ps_div:
    mov dx, 0
    div bx
```

```
    push dx
    inc cx
    cmp ax, 0
    jne ps_div
ps_out:
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    loop ps_out
    pop dx
    pop cx
    pop bx
    pop ax
    ret
print_snum endp
end main
```

Output (input: Iqra University Islamabad)

```
Type a sentence and press Enter: Iqra University Islamabad
Number of vowels = 10
```

4. EXERCISES

a) How many times does the body of the loop execute in P6-1 if `n` is 0? Correct the program using JCXZ.

b) Write a program that displays the **multiplication table** of a digit entered by the user (e.g. 7 x 1 = 7 ... 7 x 10 = 70).

c) Write a program that prints the following pattern using nested loops:

d) Write a program that computes the sum of the **even numbers** from 2 to 50 and displays it (answer: 650).

e) Write a program that reads a sentence (ending with Enter) and displays the number of **words**, **digits** and **spaces** in it.

Pattern for exercise c)

```
1
12
123
1234
12345
```

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Open-Ended Lab / Project

CISC vs RISC: Comparative Mini-Project in x86 (emu8086) and MIPS (QtSPIM)

Week / Lab Session : Assigned in Week 7, assessed in Week 16
CLO Mapping : CLO1, CLO2
Time Required : 3 hours
Software Required : emu8086 and QtSPIM

1. PROBLEM STATEMENT

The examination section wants a small **Student Result Processing** module that runs on two different processors. For a class of N students (N ≥ 10) the module must store the marks of each student in three subjects and produce: the total and percentage of every student, the grade of every student, the class average, the highest and lowest totals, and a list of students sorted by total.

Your group must implement the module **in x86 assembly (emu8086)** and **in MIPS assembly (QtSPIM)**, and then **compare the two implementations** to draw evidence-based conclusions about CISC and RISC design. There is no single correct design: you decide how data is stored, how the work is divided into procedures/macros, and how you measure and compare the programs.

2. TASKS

1. **Design (CLO1):** Choose the data layout (parallel arrays or records), the grading scheme and the procedures/macros. Draw a flowchart or pseudocode of the whole module.
2. **x86 implementation (CLO1):** Implement the module in emu8086 using procedures, macros, arrays, loops and INT 21h I/O.
3. **MIPS implementation (CLO1):** Implement the same module in QtSPIM following the MIPS calling convention and syscalls.
4. **Comparison (CLO2):** Compare the two programs: number of source lines, number of instructions, code size in bytes, number of memory accesses in the main loops, number of registers used, and ease of programming. Count executed instructions for one function in both programs.
5. **Memory system analysis (CLO2):** Extract the address trace of the sorting routine of your MIPS program and run it through the cache simulator of Lab 12. Report the hit ratio for two cache configurations.
6. **Extension (choose one):** input validation and error messages; a menu-driven interface; storing names with the marks; or a version with loop unrolling and a measurement of its effect.

3. DELIVERABLES

Deliverable	Details	Due
Proposal (1 page)	Group members, design decisions, division of work	Week 9
Source code	Both programs, well commented, with sample input files	Week 16
Report (6–10 pages)	Design, implementation, comparison tables and charts, cache results, conclusions and individual contributions	Week 16
Demonstration and viva	10-minute demo of both programs and individual questions	Week 16

4. ASSESSMENT

The project is assessed with **Lab Rubrics (Group 2)**: R1 Problem Analysis and Design, R2 Implementation and Results, R3 Demonstration and Viva, R4 Contribution/Group Participation and R5 Report and Presentation (0–3 each, total 15).

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 2). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab Session 07

1. OBJECT

Array Processing in x86 Assembly Language

Week / Lab Session : Week 9 / Lab 07
CLO Mapping : CLO1, CLO2
Time Required : 3 hours
Software Required : emu8086 (8086 microprocessor emulator)

2. THEORY

2.1 Arrays in memory

An array is a sequence of elements of the same size stored at consecutive memory addresses. It is declared by listing values or with DUP, e.g. `arr dw 45, 12, 78` (3 words) or `marks db 50 dup(0)` (50 bytes). The address of element *i* is:

address(arr[i]) = address(arr) + i × element size

So for a word array the index register is increased by 2 for each element, and for a byte array by 1.

2.2 Addressing modes for arrays

Mode	Example	Use
Indexed	mov ax, arr[si]	SI or DI holds the byte offset from the start of the array
Register indirect	lea bx, arr / mov ax, [bx]	BX points to the current element; add the element size to move on
Based-indexed	mov al, [bx+si]	BX = start of the array (or of a row), SI = position inside it
With displacement	mov al, list[si+1]	access the next element (used in sorting)

3. PROCEDURE

Program P7-1

```
; P7-1  Array processing: sum, average, maximum and minimum of a word array
.model small
.stack 100h

print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
```

```
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

.data
arr   dw 45, 12, 78, 3, 56, 91, 27, 64
len   equ 8              ; number of elements
summ  db 'Sum     = $'
avgm  db 'Average = $'
maxm  db 'Maximum = $'
minm  db 'Minimum = $'
total dw ?

.code
main proc
    mov ax, @data
    mov ds, ax

    ; ---- sum and average (SI indexes the array) ----
    mov si, 0
    mov cx, len
    mov ax, 0
sum_loop:
    add ax, arr[si]      ; based-indexed addressing
    add si, 2            ; next word = 2 bytes further
    loop sum_loop
    mov total, ax
    print summ
    call print_snum
    newline
    mov dx, 0
    mov bx, len
    div bx               ; AX = sum / len
    print avgm
    call print_snum
    newline

    ; ---- maximum and minimum (BX points to the array) ----
    lea bx, arr
    mov ax, [bx]         ; max = first element
    mov dx, [bx]         ; min = first element
    mov cx, len-1
mm_loop:
    add bx, 2
    cmp [bx], ax
    jle not_max
    mov ax, [bx]
not_max:
    cmp [bx], dx
    jge not_min
    mov dx, [bx]
not_min:
    loop mm_loop
    print maxm
```

```
    call print_snum
    newline
    mov ax, dx
    print minm
    call print_snum

    mov ah, 4Ch
    int 21h
main endp

; print_snum: display the signed number in AX in decimal
print_snum proc
    push ax
    push bx
    push cx
    push dx
    test ax, ax
    jns ps_pos
    push ax
    mov dl, '-'          ; negative: print the sign
    mov ah, 02h
    int 21h
    pop ax
    neg ax               ; make it positive (two's complement)
ps_pos:
    mov cx, 0
    mov bx, 10
ps_div:
    mov dx, 0
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne ps_div
ps_out:
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    loop ps_out
    pop dx
    pop cx
    pop bx
    pop ax
    ret
print_snum endp
end main
```

Output

```
Sum     = 376
Average = 47
Maximum = 91
Minimum = 3
```

Program P7-2

```
; P7-2  Linear search and bubble sort on a byte array
.model small
```

```
.stack 100h

print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

.data
list   db 29, 7, 18, 3, 25, 11
n      equ 6
key    db 25
foundm db 'Key 25 found at index $'
nfm    db 'Key not found$'
beforem db 'Before sort: $'
afterm db 'After sort : $'

.code
main proc
    mov ax, @data
    mov ds, ax

    ; ---- linear search ----
    mov si, 0
    mov cx, n
    mov al, key
search:
    cmp list[si], al
    je  found
    inc si
    loop search
    print nfm
    jmp search_end
found:
    print foundm
    mov ax, si
    call print_snum      ; index 4
search_end:
    newline

    print beforem
    call show_list
```

```
    newline

    ; ---- bubble sort (ascending) ----
    mov cx, n-1          ; number of passes
pass:
    push cx
    mov si, 0
compare:
    mov al, list[si]
    cmp al, list[si+1]
    jbe no_swap
    xchg al, list[si+1]  ; swap the two neighbours
    mov list[si], al
no_swap:
    inc si
    loop compare
    pop cx
    loop pass

    print afterm
    call show_list

    mov ah, 4Ch
    int 21h
main endp

; show_list: display all elements of list separated by spaces
show_list proc
    push si
    push cx
    push ax
    mov si, 0
    mov cx, n
sl_next:
    mov al, list[si]
    mov ah, 0
    call print_snum
    mov dl, ' '
    mov ah, 02h
    int 21h
    inc si
    loop sl_next
    pop ax
    pop cx
    pop si
    ret
show_list endp

; print_snum: display the signed number in AX in decimal
print_snum proc
    push ax
    push bx
    push cx
    push dx
    test ax, ax
    jns ps_pos
    push ax
    mov dl, '-'          ; negative: print the sign
    mov ah, 02h
```

```
    int 21h
    pop ax
    neg ax                ; make it positive (two's complement)
ps_pos:
    mov cx, 0
    mov bx, 10
ps_div:
    mov dx, 0
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne ps_div
ps_out:
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    loop ps_out
    pop dx
    pop cx
    pop bx
    pop ax
    ret
print_snum endp
end main
```

Output

```
Key 25 found at index 4
Before sort: 29 7 18 3 25 11
After sort : 3 7 11 18 25 29
```

Bubble sort compares neighbouring elements and swaps them when they are out of order. After the first pass the largest element reaches the end, so each later pass can be one comparison shorter; P7-2 achieves this by using the pass counter (CX) as the inner-loop count.

4. EXERCISES

a) In P7-1, why is SI increased by 2 but in P7-2 by 1? What would happen if `arr` were scanned with `add si, 1`?

b) Write a program that counts how many elements of a word array are **positive**, **negative** and **zero**. Use the array 12, −7, 0, 45, −3, 0, 8.

c) Write a program that **reverses** a byte array in place (swap the first and last elements, then the second and second-last, ...) and displays it before and after.

d) Modify P7-2 to sort the array in **descending** order and to stop early when a pass makes no swap (optimised bubble sort). Count and display the number of swaps.

e) Read a string from the keyboard (INT 21h, 0Ah) into a buffer and display it **reversed**. Also check whether it is a **palindrome** (e.g. “madam”).

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab Session 08

1. OBJECT

Development of Procedures and Macros in x86 Assembly Language

Week / Lab Session : Week 10 / Lab 08
CLO Mapping : CLO1, CLO2
Time Required : 3 hours
Software Required : emu8086 (8086 microprocessor emulator)

2. THEORY

2.1 Procedures

A **procedure** (subroutine) is a block of code that is written once and called from many places. `CALL name` pushes the return address (the offset of the next instruction) on the stack and jumps to the procedure; `RET` pops the return address into IP. A procedure should save (PUSH) and restore (POP) the registers it changes, other than those used to return results.

2.2 Passing parameters

- **In registers** – simple and fast, e.g. the number to be squared in AX (procedure `square`).
- **On the stack** – the caller pushes the parameters and calls the procedure. The procedure sets BP = SP and reads the parameters at [BP+4], [BP+6], ... After the call the caller removes them with `add sp, n` (or the procedure uses `ret n`).
- **In memory variables** – global variables (simple but less flexible).

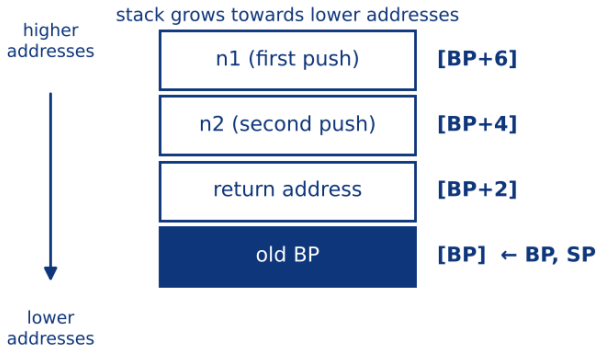


Figure 8.1: The stack frame of add_two after PUSH BP and MOV BP, SP

2.3 Recursion

A recursive procedure calls itself. Each call needs its own copy of the parameters, which are saved on the stack. In `fact`, the value *n* is pushed, `fact` is called for $n - 1$, and on return *n* is popped and multiplied by the result.

2.4 Macros versus procedures

	Macro	Procedure
Mechanism	Expanded (copied) by the assembler at each use	Exists once; called at run time with CALL/RET
Program size	Grows with every use	Does not grow with the number of calls
Speed	No call/return overhead	Small overhead for CALL, RET and stack operations
Parameters	Text substitution of arguments	Registers, stack or memory

	Macro	Procedure
Best for	Short, frequently used sequences (I/O, swap)	Longer code, code with decisions and loops, recursion

3. PROCEDURE

Program P8-1

```
; P8-1  Procedures: parameters in registers, parameters on the stack, recursion
.model small
.stack 100h

print macro msg          ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro            ; carriage return + line feed (AX, DX preserved)
    push ax
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

.data
n1   dw 150
n2   dw 275
m1   db 'square(12)        = $'
m2   db 'add_two(150, 275) = $'
m3   db 'factorial(7)      = $'

.code
main proc
    mov ax, @data
    mov ds, ax

    ; (1) parameter passed in a register
    mov ax, 12
    call square          ; returns AX = 144
    print m1
    call print_snum
    newline

    ; (2) parameters passed on the stack
    push n1
    push n2
    call add_two         ; returns AX = 425
```

```
    add sp, 4            ; caller removes the 2 parameters
    print m2
    call print_snum
    newline

    ; (3) recursive procedure
    mov ax, 7
    call fact            ; returns AX = 5040
    print m3
    call print_snum
    newline

    mov ah, 4Ch
    int 21h
main endp

; square: AX = AX * AX
square proc
    push dx
    mul ax
    pop dx
    ret
square endp

; add_two: AX = first parameter + second parameter (both on the stack)
add_two proc
    push bp
    mov bp, sp           ; [bp+2] = return address
    mov ax, [bp+4]       ; second value pushed (n2)
    add ax, [bp+6]       ; first value pushed  (n1)
    pop bp
    ret
add_two endp

; fact: AX = AX!  (recursive: n! = n * (n-1)!)
fact proc
    cmp ax, 1
    jbe fact_base        ; 0! = 1! = 1
    push ax              ; save n
    dec ax
    call fact            ; AX = (n-1)!
    pop bx               ; BX = n
    mul bx               ; AX = n * (n-1)!
    ret
fact_base:
    mov ax, 1
    ret
fact endp

; print_snum: display the signed number in AX in decimal
print_snum proc
    push ax
    push bx
    push cx
    push dx
    test ax, ax
    jns ps_pos
    push ax
    mov dl, '-'          ; negative: print the sign
```

```
    mov ah, 02h
    int 21h
    pop ax
    neg ax              ; make it positive (two's complement)
ps_pos:
    mov cx, 0
    mov bx, 10
ps_div:
    mov dx, 0
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne ps_div
ps_out:
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    loop ps_out
    pop dx
    pop cx
    pop bx
    pop ax
    ret
print_snum endp
end main
```

Output

```
square(12)        = 144
add_two(150, 275) = 425
factorial(7)      = 5040
```

1. Single-step P8-1 and open **view → stack**.
2. Before and after each CALL, note the value of SP and the return address that appears on the stack.
3. During the recursive calls of `fact`, count how many return addresses and saved values are on the stack at the deepest point.

Program P8-2

```
; P8-2  Macros with parameters versus procedures
.model small
.stack 100h

print macro msg         ; display a '$'-terminated string (AX, DX preserved)
    push ax
    push dx
    lea dx, msg
    mov ah, 09h
    int 21h
    pop dx
    pop ax
endm

newline macro           ; carriage return + line feed (AX, DX preserved)
    push ax
```

```
    push dx
    mov dl, 13
    mov ah, 02h
    int 21h
    mov dl, 10
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

; putch: display one character
putch macro char
    push ax
    push dx
    mov dl, char
    mov ah, 02h
    int 21h
    pop dx
    pop ax
endm

; swap: exchange two word variables
swap macro p, q
    push ax
    mov ax, p
    xchg ax, q
    mov p, ax
    pop ax
endm

; sum3: dest = p + q + r  (three word variables)
sum3 macro dest, p, q, r
    push ax
    mov ax, p
    add ax, q
    add ax, r
    mov dest, ax
    pop ax
endm

.data
x     dw 31
y     dw 84
z     dw 15
total dw ?
m1    db 'Before swap: X = $'
m2    db 'After swap : X = $'
my    db ', Y = $'
m3    db 'X + Y + Z  = $'

.code
main proc
    mov ax, @data
    mov ds, ax

    print m1
    mov ax, x
    call print_snum
```

```
    print my
    mov ax, y
    call print_snum
    newline

    swap x, y            ; expanded in place by the assembler

    print m2
    mov ax, x
    call print_snum
    print my
    mov ax, y
    call print_snum
    newline

    sum3 total, x, y, z  ; total = 84 + 31 + 15
    print m3
    mov ax, total
    call print_snum
    putch '!'

    mov ah, 4Ch
    int 21h
main endp

; print_snum: display the signed number in AX in decimal
print_snum proc
    push ax
    push bx
    push cx
    push dx
    test ax, ax
    jns ps_pos
    push ax
    mov dl, '-'          ; negative: print the sign
    mov ah, 02h
    int 21h
    pop ax
    neg ax               ; make it positive (two's complement)
ps_pos:
    mov cx, 0
    mov bx, 10
ps_div:
    mov dx, 0
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne ps_div
ps_out:
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
    loop ps_out
    pop dx
    pop cx
    pop bx
    pop ax
```

```
    ret
print_snum endp
end main
```

Output

```
Before swap: X = 31, Y = 84
After swap : X = 84, Y = 31
X + Y + Z  = 130!
```

4. EXERCISES

a) For P8-1, draw the stack (addresses and contents) at the moment `mov ax, [bp+4]` executes in `add_two`.

b) Rewrite `add_two` so that it uses `ret 4` to remove the parameters. What change is needed in the caller?

c) Write a procedure `power` that receives a base and an exponent on the stack and returns base^exponent in AX. Test it with 3^4 and 2^10.

d) Write a **recursive** procedure that computes the n-th Fibonacci number and display the first 10 Fibonacci numbers.

e) Write a macro `print_char_n char, n` that displays a character n times, and a procedure that does the same. Use each 3 times in a program and compare the size of the generated code in the emulator.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab Session 09

1. OBJECT

Familiarization with SPIM – a MIPS simulator

Week / Lab Session : Week 11 / Lab 09
CLO Mapping : CLO1, CLO2
Time Required : 3 hours
Software Required : QtSPIM (MIPS32 simulator)

2. THEORY

2.1 The MIPS architecture

MIPS is a **RISC** (Reduced Instruction Set Computer) architecture. Compared with the CISC x86 it has: a small set of simple instructions, all **32 bits long**; **32 general-purpose 32-bit registers**; a **load/store** design in which only `lw`/`sw` (and their byte/half variants) access memory while all arithmetic works on registers; and a few regular instruction formats that are easy to decode and pipeline.

Feature	x86 (CISC)	MIPS (RISC)
Instruction length	Variable (1 to 15 bytes)	Fixed (4 bytes)
General-purpose registers	8 (AX, BX, ... in 16-bit mode)	32
Memory operands	Most instructions can access memory	Only load and store instructions
Operands per instruction	Usually 2 (destination is also a source)	Usually 3 (rd, rs, rt)
Byte order	Little endian	Either; SPIM uses the host's order (little endian on PCs)

2.2 MIPS registers

Name	Number	Usage
$zero	0	Constant value 0
$at	1	Reserved for the assembler (pseudo-instructions)
$v0 – $v1	2–3	Values returned by procedures; syscall number in $v0
$a0 – $a3	4–7	Arguments to procedures and syscalls
$t0 – $t7	8–15	Temporaries (not preserved across calls)
$s0 – $s7	16–23	Saved registers (preserved across calls)
$t8 – $t9	24–25	More temporaries
$k0 – $k1	26–27	Reserved for the OS kernel
$gp	28	Global pointer
$sp	29	Stack pointer
$fp	30	Frame pointer
$ra	31	Return address (set by jal)

In addition, **PC** holds the address of the current instruction and **HI** and **LO** hold the results of multiply and divide.

2.3 Instruction formats

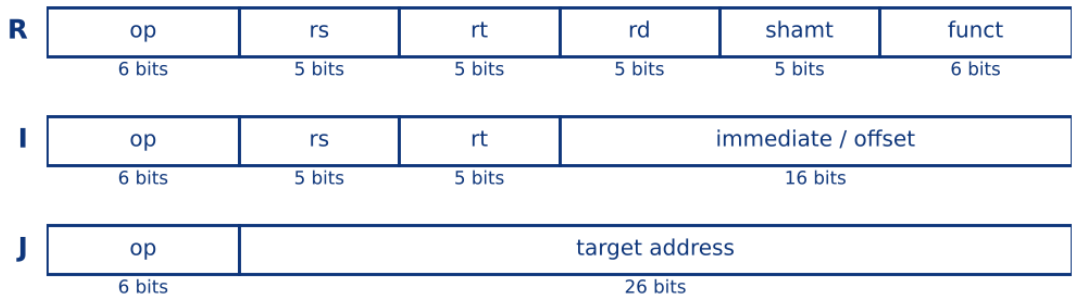


Figure 9.1: The three MIPS instruction formats

Format	Used for	Example
R-type	Register–register operations	add $t2, $t0, $t1 ($t2 = $t0 + $t1)
I-type	Immediate operands, loads/stores, branches	addi $t4, $t0, -5 / lw $t0, 8($s0) / beq $t0, $t1, label
J-type	Jumps	j loop / jal max

2.4 Program structure and memory layout

A SPIM program has a **data segment** (`.data`) for variables and a **text segment** (`.text`) for instructions. Execution starts at the label `main`, which must be declared global with `.globl main`. Common data directives are `.word` (32-bit), `.half`, `.byte`, `.asciiz` (string ending in a null byte), `.ascii` and `.space n` (n bytes).

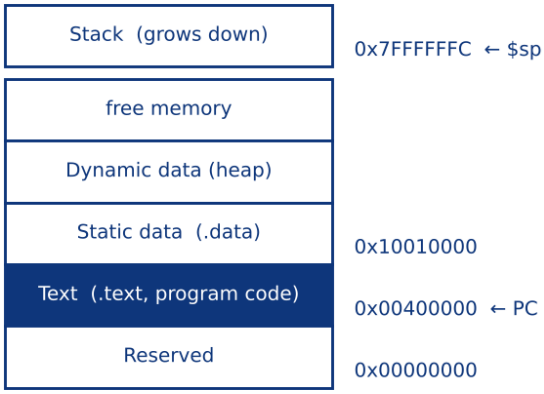


Figure 9.2: Memory layout of a program in SPIM

SPIM also accepts **pseudo-instructions**, which the assembler translates into one or more real instructions using register $at. For example `li $t0, 25` becomes `ori $8, $0, 25`, and `move $a0, $t2` becomes `addu $4, $0, $10`. You can see the translation in the text segment window.

3. PROCEDURE

Program P9-1

```
# P9-1  First MIPS program: arithmetic and logical instructions on registers
        .text
        .globl main
main:
        li    $t0, 25          # $t0 = 25          (pseudo-instruction: addiu)
        li    $t1, 12          # $t1 = 12
        add   $t2, $t0, $t1    # $t2 = 25 + 12 = 37
```

```
        sub   $t3, $t0, $t1    # $t3 = 25 - 12 = 13
        addi  $t4, $t0, -5     # $t4 = 25 - 5  = 20   (immediate operand)
        and   $t5, $t0, $t1    # $t5 = 11001 AND 01100 = 01000 = 8
        or    $t6, $t0, $t1    # $t6 = 11001 OR  01100 = 11101 = 29
        sll   $t7, $t1, 2      # $t7 = 12 << 2 = 48  (multiply by 4)

        li    $v0, 1           # syscall 1: print integer in $a0
        move  $a0, $t2
        syscall
        li    $v0, 11          # syscall 11: print character in $a0
        li    $a0, ' '
        syscall
        li    $v0, 1
        move  $a0, $t7
        syscall

        li    $v0, 10          # syscall 10: exit
        syscall
```

Output

```
37 48
```

1. Type P9-1 in a text editor and save it as `lab9_p1.asm`.
2. Start QtSPIM and load the file with **File → Reinitialize and Load File**.
3. In the **text segment** window, find the instructions of `main` (starting near address 0x00400024). Note the machine code of each instruction and how pseudo-instructions were translated.
4. Press **F10 (single step)** repeatedly and watch registers R8–R15 ($t0–$t7) change. Select **Registers → Decimal** to view the values in decimal.
5. Press **F5** to run the program to the end and look at the console window.

Program P9-2

```
# P9-2  Memory access with load/store: z = x + y - 5
        .data
x:      .word 120
y:      .word -45
z:      .word 0
msg:    .asciiz "z = x + y - 5 = "

        .text
        .globl main
main:
        lw    $t0, x           # $t0 = Mem[x] = 120
        lw    $t1, y           # $t1 = Mem[y] = -45
        add   $t2, $t0, $t1    # $t2 = 75
        addi  $t2, $t2, -5     # $t2 = 70
        sw    $t2, z           # Mem[z] = 70

        li    $v0, 4           # print the message
        la    $a0, msg
        syscall
        li    $v0, 1           # print z (read back from memory)
        lw    $a0, z
        syscall

        li    $v0, 10
```

```
        syscall
```

Output

```
z = x + y - 5 = 70
```

In the **data segment** window, the variables start at address 0x10010000: x = 0x00000078 (120), y = 0xfffffd3 (−45 in two's complement) and, after the `sw` instruction, z = 0x00000046 (70).

4. EXERCISES

a) For P9-1, record the machine code of the following instructions from the text segment window and identify the format and fields.

Instruction	Machine code (hex)	Format	op / rs / rt / rd / shamt / funct or immediate
add $t2, $t0, $t1			
sub $t3, $t0, $t1			
addi $t4, $t0, -5			
sll $t7, $t1, 2			

b) Which instructions of P9-1 are pseudo-instructions? Write the real instructions that SPIM generated for each.

c) Modify P9-2 to compute `z = (x + y) × 4 − (x − y)` using `sll` for the multiplication. Verify the value of z in the data segment window.

d) Declare a variable `val: .word 0x12345678`. Use `lb` to load the byte at address `val` into $t0. Which byte do you get? What does this tell you about the byte order used by SPIM on your computer?

e) Compare P9-2 with program P2-1 of Lab 02 (x86). List three differences between the two ISAs that you can see in the programs.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab Session 10

1. OBJECT

Learning use of SPIM console and system calls; Developing Procedures in MIPS Assembly Language

Week / Lab Session : Week 12 / Lab 10
CLO Mapping : CLO1, CLO2
Time Required : 3 hours
Software Required : QtSPIM (MIPS32 simulator)

2. THEORY

2.1 System calls

SPIM provides a small set of operating-system services through the `syscall` instruction. The service number is placed in **$v0** and the arguments in **$a0–$a3** (or $f12 for floating point); results are returned in **$v0**.

$v0	Service	Arguments	Result
1	print_int	$a0 = integer	–
4	print_string	$a0 = address of null-terminated string	–
5	read_int	–	$v0 = integer read
8	read_string	$a0 = buffer address, $a1 = buffer length	string stored in the buffer
9	sbrk (allocate memory)	$a0 = number of bytes	$v0 = address of the block
10	exit	–	program ends
11	print_char	$a0 = character	–
12	read_char	–	$v0 = character read

2.2 Procedures in MIPS

MIPS has no CALL/RET or PUSH/POP instructions. Procedures follow a software **calling convention**:

- **`jal label`** (jump and link) saves the return address (PC + 4) in **$ra** and jumps to the procedure; **`jr $ra`** returns.
- Arguments are passed in **$a0–$a3**; results are returned in **$v0–$v1**.
- The procedure may freely change $t registers but must preserve **$s0–$s7**, $sp and $ra for its caller.
- A **leaf** procedure (one that calls no other procedure) often needs no stack. A **non-leaf** or recursive procedure must save $ra (and any argument it still needs) on the **stack** because `jal` overwrites $ra.
- The stack grows towards lower addresses: allocate with `addi $sp, $sp, -8`, save with `sw $ra, 4($sp)`, restore with `lw`, and release with `addi $sp, $sp, 8`.

3. PROCEDURE

Program P10-1

```
# P10-1  SPIM system calls: read integers and a string, print results
        .data
ask1:   .asciiz "Enter first integer : "
ask2:   .asciiz "Enter second integer: "
askn:   .asciiz "Enter your name     : "
```

```
summ:   .asciiz "Sum = "
prodm:  .asciiz "\nProduct = "
hello:  .asciiz "Welcome to the MIPS lab, "
name:   .space 32              # buffer for the name (32 bytes)

        .text
        .globl main
main:
        li    $v0, 4           # print string
        la    $a0, ask1
        syscall
        li    $v0, 5           # read integer -> $v0
        syscall
        move  $s0, $v0

        li    $v0, 4
        la    $a0, ask2
        syscall
        li    $v0, 5
        syscall
        move  $s1, $v0

        li    $v0, 4
        la    $a0, summ
        syscall
        add   $a0, $s0, $s1
        li    $v0, 1           # print integer
        syscall

        li    $v0, 4
        la    $a0, prodm
        syscall
        mul   $a0, $s0, $s1    # pseudo-instruction (mult + mflo)
        li    $v0, 1
        syscall

        li    $v0, 11          # print character: newline
        li    $a0, 10          # ASCII 10 = newline
        syscall

        li    $v0, 4
        la    $a0, askn
        syscall
        li    $v0, 8           # read string: $a0 = buffer, $a1 = length
        la    $a0, name
        li    $a1, 32
        syscall

        li    $v0, 4
        la    $a0, hello
        syscall
        li    $v0, 4
        la    $a0, name
        syscall

        li    $v0, 10
        syscall
```

Console (inputs: 15, 8, Sara Ahmed)

```
Enter first integer : 15
Enter second integer: 8
Sum = 23
Product = 120
Enter your name     : Sara Ahmed
Welcome to the MIPS lab, Sara Ahmed
```

Program P10-2

```
# P10-2  Procedures in MIPS: a leaf procedure and a recursive procedure
        .data
m1:     .asciiz "max(17, 42) = "
m2:     .asciiz "\nfactorial(6) = "

        .text
        .globl main
main:
        li    $a0, 17          # arguments in $a0, $a1
        li    $a1, 42
        jal   max              # $ra = return address, jump to max
        move  $s0, $v0         # result returned in $v0

        li    $v0, 4
        la    $a0, m1
        syscall
        li    $v0, 1
        move  $a0, $s0
        syscall

        li    $a0, 6
        jal   fact
        move  $s0, $v0
        li    $v0, 4
        la    $a0, m2
        syscall
        li    $v0, 1
        move  $a0, $s0
        syscall

        li    $v0, 10
        syscall

# ---- max(a, b): leaf procedure, needs no stack frame ----
max:
        move  $v0, $a0
        bge   $a0, $a1, max_done   # if a >= b return a
        move  $v0, $a1             # else return b
max_done:
        jr    $ra

# ---- fact(n) = n * fact(n-1): non-leaf, saves $ra and $a0 on the stack ----
fact:
        addi  $sp, $sp, -8     # make room for 2 words
        sw    $ra, 4($sp)      # save return address
        sw    $a0, 0($sp)      # save argument n
        li    $t0, 2
        bge   $a0, $t0, fact_rec
        li    $v0, 1           # base case: n < 2 -> 1
```

```
        addi  $sp, $sp, 8
        jr    $ra
fact_rec:
        addi  $a0, $a0, -1
        jal   fact            # $v0 = fact(n-1)
        lw    $a0, 0($sp)     # restore n
        lw    $ra, 4($sp)     # restore return address
        addi  $sp, $sp, 8     # pop the stack frame
        mul   $v0, $a0, $v0   # n * fact(n-1)
        jr    $ra
```

Output

```
max(17, 42) = 42
factorial(6) = 720
```

1. Load P10-2 and single-step it. Before and after each `jal`, record the values of PC and $ra.
2. During the recursive calls of `fact`, watch $sp decrease by 8 for every call. Open the data segment window at the stack (address near 0x7fffeffc) to see the saved $ra and $a0 values.

4. EXERCISES

a) Fill in the stack contents when `fact` is called with n = 3 at the deepest point of the recursion.

Address ($sp + ...)	Contents	Saved by call fact(n) with n =

b) Write a program that reads an integer and displays whether it is **even or odd** and **positive, negative or zero**.

c) Write a leaf procedure `sum3` that receives three integers in $a0–$a2 and returns their sum in $v0. Call it from `main` with values read from the keyboard.

d) Write a recursive procedure `power(base, exp)` and test it with power(2, 10) = 1024.

e) Write a program that reads a string with syscall 8 and counts its **vowels** (compare with program P6-3 in x86). Use `lb` to read one character at a time and stop at the newline or null character.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab Session 11

1. OBJECT

Implementing vector operations in MIPS Assembly and exploring Loop Unrolling

Week / Lab Session : Week 13 / Lab 11
CLO Mapping : CLO1, CLO2
Time Required : 3 hours
Software Required : QtSPIM (MIPS32 simulator)

2. THEORY

2.1 Vectors (arrays) in MIPS

A vector of N words occupies 4N consecutive bytes. To access element *i*, compute the byte offset `i × 4` (with `sll $t1, $t0, 2`), add it to the base address and use `lw`/`sw`. Alternatively, keep a **pointer** register and increase it by 4 in each iteration. Common vector operations are element-wise addition **C = A + B**, scaling **C = kA**, and the **dot product** A·B = Σ A[i]·B[i].

2.2 Loop unrolling

Every loop iteration executes **overhead** instructions in addition to the useful work: updating the index or pointer, and the conditional branch. **Loop unrolling** replicates the loop body k times and adjusts the loop control, so the overhead is paid once per k elements:

	Rolled loop	Unrolled by 4
Elements per iteration	1	4
Instructions per iteration	4 (lw, add, addi, bne)	10 (4 lw, 4 add, addi, bne)
Iterations for 16 elements	16	4
Loop-body instructions executed	16 × 4 = 64	4 × 10 = 40

Unrolling also exposes more independent instructions that a pipelined or superscalar processor can overlap. Its costs are larger code, more registers, and the need for extra “cleanup” code when N is not a multiple of k. Compilers perform unrolling automatically at higher optimisation levels.

3. PROCEDURE

Program P11-1

```
# P11-1  Vector operations: C = A + B, dot product A.B, scaling 3*A
        .data
A:      .word 2, 4, 6, 8, 10, 12, 14, 16
B:      .word 1, 3, 5, 7, 9, 11, 13, 15
C:      .space 32              # 8 words x 4 bytes
N:      .word 8
mC:     .asciiz "C = A + B : "
mDot:   .asciiz "\nA . B     = "

        .text
        .globl main
main:
        la    $s0, A           # base addresses
        la    $s1, B
        la    $s2, C
```

```
        lw    $s3, N           # number of elements
        li    $t0, 0           # i = 0
        li    $s4, 0           # dot = 0
loop:
        beq   $t0, $s3, done   # while (i < N)
        sll   $t1, $t0, 2      # byte offset = i * 4
        add   $t2, $s0, $t1
        lw    $t3, 0($t2)      # A[i]
        add   $t2, $s1, $t1
        lw    $t4, 0($t2)      # B[i]
        add   $t5, $t3, $t4    # A[i] + B[i]
        add   $t2, $s2, $t1
        sw    $t5, 0($t2)      # C[i] = A[i] + B[i]
        mul   $t6, $t3, $t4
        add   $s4, $s4, $t6    # dot += A[i] * B[i]
        addi  $t0, $t0, 1      # i++
        j     loop
done:
        li    $v0, 4
        la    $a0, mC
        syscall
        li    $t0, 0
print:                         # print the elements of C
        beq   $t0, $s3, pdone
        sll   $t1, $t0, 2
        add   $t2, $s2, $t1
        lw    $a0, 0($t2)
        li    $v0, 1
        syscall
        li    $v0, 11
        li    $a0, 32          # space
        syscall
        addi  $t0, $t0, 1
        j     print
pdone:
        li    $v0, 4
        la    $a0, mDot
        syscall
        li    $v0, 1
        move  $a0, $s4
        syscall
        li    $v0, 10
        syscall
```

Output

```
C = A + B : 3 7 11 15 19 23 27 31
A . B     = 744
```

Program P11-2

```
# P11-2  Loop unrolling: sum of a 16-element array, rolled vs unrolled by 4
        .data
arr:    .word 5, 9, 2, 7, 4, 8, 1, 6, 3, 10, 12, 11, 15, 13, 14, 16
m1:     .asciiz "Rolled loop sum    = "
m2:     .asciiz "\nUnrolled loop sum  = "
m3:     .asciiz "\nInstructions in loop body executed (rolled / unrolled): "
m4:     .asciiz " / "
```

```
        .text
        .globl main
main:
        # ---------- rolled loop: 1 element per iteration ----------
        la    $t0, arr         # pointer
        addi  $t1, $t0, 64     # end address = arr + 16*4
        li    $s0, 0           # sum
        li    $s2, 0           # instruction counter (for measurement only)
rolled:
        lw    $t2, 0($t0)
        add   $s0, $s0, $t2
        addi  $t0, $t0, 4
        addi  $s2, $s2, 4      # 4 instructions in this body (incl. bne)
        bne   $t0, $t1, rolled

        # ---------- unrolled loop: 4 elements per iteration ----------
        la    $t0, arr
        li    $s1, 0
        li    $s3, 0
unrolled:
        lw    $t2, 0($t0)
        lw    $t3, 4($t0)
        lw    $t4, 8($t0)
        lw    $t5, 12($t0)
        add   $s1, $s1, $t2
        add   $s1, $s1, $t3
        add   $s1, $s1, $t4
        add   $s1, $s1, $t5
        addi  $t0, $t0, 16     # advance by 4 elements
        addi  $s3, $s3, 10     # 10 instructions in this body (incl. bne)
        bne   $t0, $t1, unrolled

        li    $v0, 4
        la    $a0, m1
        syscall
        li    $v0, 1
        move  $a0, $s0
        syscall
        li    $v0, 4
        la    $a0, m2
        syscall
        li    $v0, 1
        move  $a0, $s1
        syscall
        li    $v0, 4
        la    $a0, m3
        syscall
        li    $v0, 1
        move  $a0, $s2
        syscall
        li    $v0, 4
        la    $a0, m4
        syscall
        li    $v0, 1
        move  $a0, $s3
        syscall
        li    $v0, 10
        syscall
```

Output

```
Rolled loop sum    = 136
Unrolled loop sum  = 136
Instructions in loop body executed (rolled / unrolled): 64 / 40
```

The registers $s2 and $s3 in P11-2 only **count** the loop-body instructions so that the effect of unrolling can be displayed; they are not part of the algorithm. Single-step both loops and confirm the number of iterations.

4. EXERCISES

a) Modify P11-1 to compute **C = 3A − B** and display C.

b) Rewrite the loop of P11-1 using a **pointer** (increment the addresses by 4) instead of computing `i × 4`. How many instructions per iteration are saved?

c) Unroll the dot-product loop of P11-1 by a factor of 2. Count the loop-body instructions executed before and after unrolling.

d) Modify P11-2 for an array of 18 elements and unroll by 4. Add the cleanup code needed for the remaining 2 elements and verify the sum.

e) Complete the table for an array of N = 64 elements. What is the percentage reduction in executed loop-body instructions for each factor?

Unroll factor	Instructions per iteration	Iterations	Instructions executed	Reduction (%)
1	4	64	256	–
2				
4				
8				

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab Session 12

1. OBJECT

Simulating Cache Read/Write Operations

Week / Lab Session	:	Week 14 / Lab 12
CLO Mapping	:	CLO1, CLO2
Time Required	:	3 hours
Software Required	:	QtSPIM (MIPS32 simulator)

2. THEORY

2.1 Cache memory

A **cache** is a small, fast memory between the processor and main memory that holds copies of recently used data. It works because programs show **locality of reference**: *temporal locality* (a location used now is likely to be used again soon) and *spatial locality* (nearby locations are likely to be used soon). Data moves between main memory and the cache in **blocks** (lines) of several bytes.

- **Hit** – the requested data is in the cache. **Miss** – it is not, and the block must be fetched from main memory.
- **Hit ratio** = hits / total accesses. **Average memory access time** = hit time + miss rate × miss penalty.
- Misses are classified as **compulsory** (first access to a block), **capacity** (the cache is too small) and **conflict** (two blocks compete for the same line).

2.2 Direct-mapped cache

In a direct-mapped cache each memory block can be placed in exactly one line: **line = block number mod number of lines**. The address is divided into three fields:

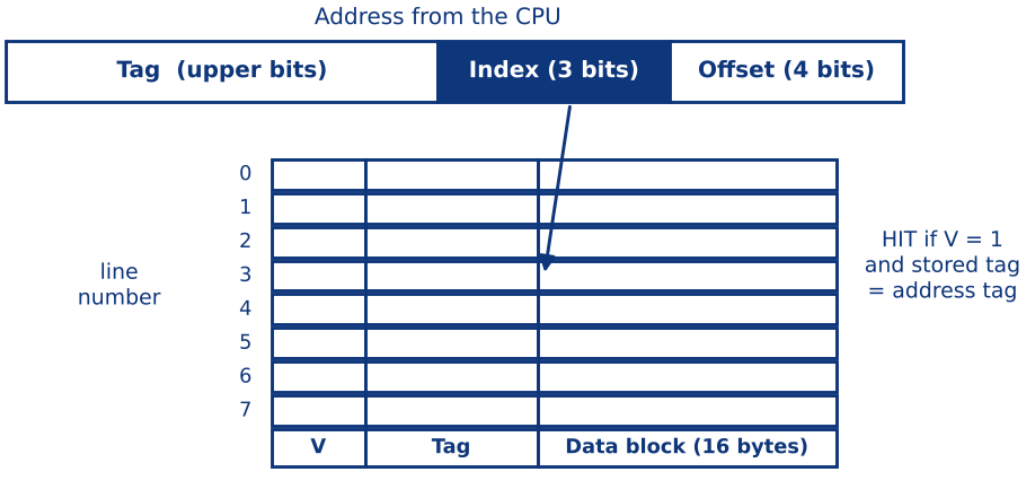


Figure 12.1: Address fields and organisation of an 8-line direct-mapped cache with 16-byte blocks

- **Offset** = log2(block size) bits: the byte inside the block (16-byte block → 4 bits).
- **Index** = log2(number of lines) bits: selects the cache line (8 lines → 3 bits).
- **Tag** = the remaining upper bits: identifies which block is currently stored in the line.

A line also has a **valid bit**. An access is a hit when the line is valid and its stored tag equals the tag of the address. **Example:** address 132 = 1000 0100 (binary) → offset = 0100 = 4, index = 000 = 0, tag = 1. Address 0 also maps to line 0 (tag 0), so addresses 0 and 132 cause **conflict misses** when used alternately.

Set-associative caches reduce conflicts by allowing a block to go into any of the k lines of a set (k-way), and a **fully associative** cache allows any line; a replacement policy (e.g. LRU) then chooses the victim.

2.3 Write policies

Policy	On a write hit	On a write miss
Write-through	Write to the cache and to main memory	Usually no-write-allocate: write to memory only
Write-back	Write to the cache only and set the line's dirty bit; memory is updated when the line is replaced	Usually write-allocate: fetch the block, then write in the cache

3. PROCEDURE

Program P12-1 is a **cache simulator written in MIPS**. It keeps a valid bit and a tag for each of the 8 lines of a direct-mapped cache with 16-byte blocks, processes a trace of byte addresses, and reports a hit or a miss for every access.

Program P12-1

```
# P12-1  Direct-mapped cache simulator
# Cache: 8 lines, block size 16 bytes  ->  offset = 4 bits, index = 3 bits,
#        tag = remaining upper bits.   Address trace is read from memory.
        .data
trace:  .word 0, 4, 16, 20, 132, 0, 36, 40, 132, 16   # byte addresses
N:      .word 10
valid:  .space 32              # 8 valid bits (one word each)
tags:   .space 32              # 8 tags
hdr:    .asciiz "Address\tIndex\tTag\tResult\n"
hitm:   .asciiz "\tHIT\n"
missm:  .asciiz "\tMISS\n"
sp2:    .asciiz "\t"
resm:   .asciiz "\nHits = "
ofm:    .asciiz " out of "

        .text
        .globl main
main:
        li    $v0, 4
        la    $a0, hdr
        syscall
        la    $s0, trace
        lw    $s1, N
        li    $s2, 0           # i
        li    $s3, 0           # hit counter
next:
        beq   $s2, $s1, finish
        sll   $t0, $s2, 2
        add   $t0, $s0, $t0
        lw    $t1, 0($t0)      # $t1 = address
        srl   $t2, $t1, 4      # drop the 4 offset bits
        andi  $t3, $t2, 7      # index = next 3 bits
        srl   $t4, $t2, 3      # tag   = the rest

        move  $a0, $t1         # print address, index and tag
        li    $v0, 1
        syscall
        li    $v0, 4
        la    $a0, sp2
        syscall
```

```
        move  $a0, $t3
        li    $v0, 1
        syscall
        li    $v0, 4
        la    $a0, sp2
        syscall
        move  $a0, $t4
        li    $v0, 1
        syscall

        sll   $t5, $t3, 2      # word offset of this cache line
        la    $t6, valid
        add   $t6, $t6, $t5
        la    $t7, tags
        add   $t7, $t7, $t5
        lw    $t8, 0($t6)      # valid bit
        lw    $t9, 0($t7)      # stored tag
        beq   $t8, $zero, miss # line empty -> miss
        bne   $t9, $t4, miss   # different tag -> miss (conflict)
        addi  $s3, $s3, 1      # hit
        li    $v0, 4
        la    $a0, hitm
        syscall
        j     advance
miss:
        li    $t8, 1           # bring the block into the cache
        sw    $t8, 0($t6)
        sw    $t4, 0($t7)
        li    $v0, 4
        la    $a0, missm
        syscall
advance:
        addi  $s2, $s2, 1
        j     next
finish:
        li    $v0, 4
        la    $a0, resm
        syscall
        move  $a0, $s3
        li    $v0, 1
        syscall
        li    $v0, 4
        la    $a0, ofm
        syscall
        move  $a0, $s1
        li    $v0, 1
        syscall
        li    $v0, 10
        syscall
```

Output

```
Address Index   Tag     Result
0       0       0       MISS
4       0       0       HIT
16      1       0       MISS
20      1       0       HIT
132     0       1       MISS
0       0       0       MISS
```

```
36      2       0       MISS
40      2       0       HIT
132     0       1       MISS
16      1       0       HIT

Hits = 4 out of 10
```

1. Load and run P12-1 and compare the output with your own hand-traced result.
2. Single-step one access and identify where the index and the tag are extracted (`srl` and `andi`).
3. Explain each miss as compulsory or conflict. (Addresses 132 and 0 both map to line 0 and evict each other.)

4. EXERCISES

a) Hand-trace the following address trace for the cache of P12-1 (8 lines, 16-byte blocks) and then verify it by changing `trace` in the program.

Address	Index	Tag	Hit / Miss	Miss type
8				
24				
140				
12				
256				
28				
8				

b) Change the simulator to a cache of **16 lines** with **8-byte blocks**. Which shift amounts and masks must change? Run the original trace again and compare the hit ratio.

c) Extend the simulator to **write operations** using a write-back policy: add a second array giving the access type (0 = read, 1 = write) and a **dirty** bit per line. Count the number of write-backs to memory.

d) Generate the address trace of a program that sums a 16-element word array starting at address 0 (addresses 0, 4, 8, ..., 60). What hit ratio does the cache give, and which type of locality does it exploit?

e) Explain how a **2-way set-associative** cache with the same total size would handle the trace in P12-1. How many misses would remain?

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab Session 13

1. OBJECT

Learning Address Translation in a Virtual Memory System

Week / Lab Session	:	Week 15 / Lab 13
CLO Mapping	:	CLO1, CLO2
Time Required	:	3 hours
Software Required	:	QtSPIM (MIPS32 simulator)

2. THEORY

2.1 Virtual memory

Virtual memory gives every program its own large, private address space that is independent of the amount of physical memory. Programs use **virtual addresses**; the hardware (memory management unit, MMU) and the operating system translate them into **physical addresses**. Benefits include protection between programs, running programs larger than physical memory, and simpler loading and sharing.

2.2 Paging and address translation

The virtual address space is divided into fixed-size **pages** and physical memory into **frames** of the same size (e.g. 4 KB). A virtual address is split into a **virtual page number (VPN)** and a **page offset**. The **page table** maps each VPN to a frame number and has a **valid bit** that tells whether the page is in physical memory.

Physical address = (frame number × page size) + offset i.e. frame number followed by the unchanged offset bits

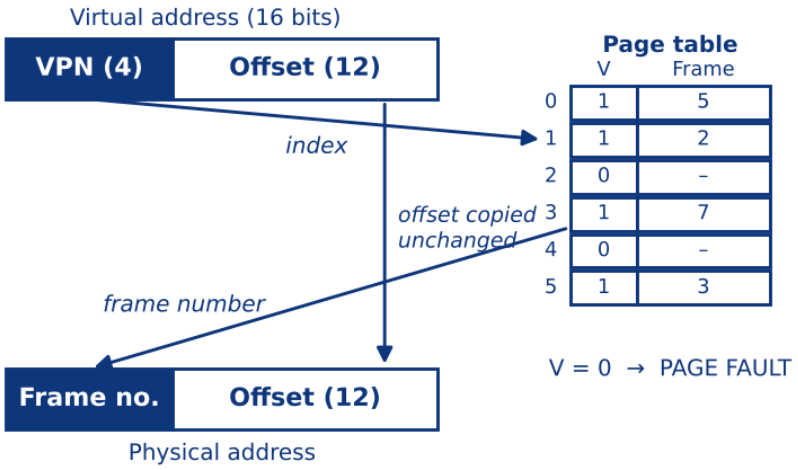


Figure 13.1: Translating a 16-bit virtual address with 4 KB pages

If the valid bit is 0, the access causes a **page fault**: the operating system loads the page from disk into a free frame (replacing another page if necessary, e.g. using LRU), updates the page table and restarts the instruction. Because every memory access needs a page-table look-up, processors keep recently used translations in a small cache called the **TLB (Translation Look-aside Buffer)**.

Example: virtual address 0x1A3C with 4 KB pages (12-bit offset) → VPN = 0x1, offset = 0xA3C. If page 1 is in frame 2, the physical address is 0x2A3C.

3. PROCEDURE

Program P13-1 simulates the MMU for a 16-bit virtual address space with 4 KB pages (16 pages). The page table is stored as two arrays (valid bits and frame numbers). The procedure `print_hex` displays numbers in hexadecimal.

Program P13-1

```
# P13-1  Virtual-to-physical address translation with a page table
# 16-bit virtual address, page size 4 KB  ->  VPN = upper 4 bits,
# offset = lower 12 bits.  The page table has 16 entries (valid bit, frame no.).
        .data
valid:  .word 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1
frame:  .word 5, 2, 0, 7, 0, 3, 0, 0, 1, 0, 0, 0, 0, 0, 0, 6
vaddr:  .word 0x1A3C, 0x0004, 0x3FFF, 0x2100, 0x5123, 0x8ABC, 0xF00F, 0x7777
N:      .word 8
hdr:    .asciiz "VA    VPN OFFSET -> PA\n"
tab:    .asciiz "  "
tab3:   .asciiz "   "
arrow:  .asciiz "    -> "
fault:  .asciiz "PAGE FAULT\n"
nl:     .asciiz "\n"
faults: .asciiz "\nPage faults = "

        .text
        .globl main
main:
        li    $v0, 4
        la    $a0, hdr
        syscall
        la    $s0, vaddr
        lw    $s1, N
        li    $s2, 0           # i
        li    $s3, 0           # page-fault counter
next:
        beq   $s2, $s1, finish
        sll   $t0, $s2, 2
        add   $t0, $s0, $t0
        lw    $s4, 0($t0)      # $s4 = virtual address
        srl   $s5, $s4, 12     # VPN    = VA >> 12
        andi  $s6, $s4, 0x0FFF # offset = VA & 0xFFF

        move  $a0, $s4         # print VA, VPN, offset in hex
        li    $a1, 4
        jal   print_hex
        li    $v0, 4
        la    $a0, tab
        syscall
        move  $a0, $s5
        li    $a1, 1
        jal   print_hex
        li    $v0, 4
        la    $a0, tab3
        syscall
        move  $a0, $s6
        li    $a1, 3
        jal   print_hex
        li    $v0, 4
        la    $a0, arrow
        syscall
```

```
        sll   $t1, $s5, 2      # look up the page-table entry
        la    $t2, valid
        add   $t2, $t2, $t1
        lw    $t3, 0($t2)
        beq   $t3, $zero, pf   # valid bit = 0 -> page fault
        la    $t2, frame
        add   $t2, $t2, $t1
        lw    $t4, 0($t2)      # frame number
        sll   $t4, $t4, 12     # PA = frame << 12 | offset
        or    $a0, $t4, $s6
        li    $a1, 4
        jal   print_hex
        li    $v0, 4
        la    $a0, nl
        syscall
        j     advance
pf:
        addi  $s3, $s3, 1
        li    $v0, 4
        la    $a0, fault
        syscall
advance:
        addi  $s2, $s2, 1
        j     next
finish:
        li    $v0, 4
        la    $a0, faults
        syscall
        li    $v0, 1
        move  $a0, $s3
        syscall
        li    $v0, 10
        syscall

# print_hex: print the lowest $a1 hex digits of $a0 (leaf procedure)
print_hex:
        move  $t8, $a0
        addi  $t9, $a1, -1     # digit position, from most significant
ph_loop:
        sll   $t7, $t9, 2      # shift = position * 4
        srlv  $t6, $t8, $t7
        andi  $t6, $t6, 0xF
        addi  $a0, $t6, 48     # '0'..'9'
        blt   $t6, 10, ph_put
        addi  $a0, $t6, 55     # 'A'..'F'
ph_put:
        li    $v0, 11
        syscall
        addi  $t9, $t9, -1
        bge   $t9, $zero, ph_loop
        jr    $ra
```

Output

```
VA    VPN OFFSET -> PA
1A3C  1   A3C    -> 2A3C
0004  0   004    -> 5004
3FFF  3   FFF    -> 7FFF
2100  2   100    -> PAGE FAULT
```

```
5123  5   123    -> 3123
8ABC  8   ABC    -> 1ABC
F00F  F   00F    -> 600F
7777  7   777    -> PAGE FAULT

Page faults = 2
```

1. Load and run P13-1. Check each translation by hand using the page table in the data segment.
2. Single-step the translation of one address and identify the instructions that extract the VPN (`srl`) and the offset (`andi`) and build the physical address (`sll`, `or`).

4. EXERCISES

a) Using the page table of P13-1, translate the following virtual addresses by hand and verify with the program.

Virtual address	VPN	Offset	Valid?	Physical address / page fault
0x0FFF				
0x3001				
0x6ABC				
0xF123				
0x5000				

b) For a 32-bit virtual address with 4 KB pages, how many bits are used for the VPN and the offset? How many entries does a single-level page table need, and how large is it if each entry is 4 bytes?

c) Modify P13-1 to use a page size of **1 KB** (10-bit offset, 64 pages). Which instructions change?

d) Extend P13-1 to handle page faults: when a page is not valid, load it into the next free frame (frames 8, 9, 10, ...), set its valid bit and print "PAGE FAULT – loaded into frame n". Run the trace twice and compare the number of faults.

e) Add a **4-entry TLB** (arrays of VPNs and frames) to the program. Check the TLB before the page table, count TLB hits and misses, and display the TLB hit ratio.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

APPENDIX A: x86 / emu8086 QUICK REFERENCE

A.1 DOS INT 21h services used in this workbook

AH	Service	Input	Output
01h	Read character with echo	–	AL = ASCII code
02h	Display character	DL = character	–
08h	Read character without echo	–	AL = ASCII code
09h	Display '$'-terminated string	DS:DX = string	–
0Ah	Buffered string input	DS:DX = buffer (byte 0 = max)	byte 1 = length, bytes 2.. = text
4Ch	Terminate program	AL = return code	–

A.2 Useful ASCII codes

Character	Hex	Decimal	Character	Hex	Decimal
Carriage return (CR)	0Dh	13	'0' – '9'	30h – 39h	48 – 57
Line feed (LF)	0Ah	10	'A' – 'Z'	41h – 5Ah	65 – 90
Space	20h	32	'a' – 'z'	61h – 7Ah	97 – 122
'$'	24h	36	'-'	2Dh	45

A.3 Frequently used x86 instructions

Category	Instructions
Data transfer	MOV, XCHG, LEA, PUSH, POP, XLAT, IN, OUT
Arithmetic	ADD, ADC, SUB, SBB, INC, DEC, NEG, CMP, MUL, IMUL, DIV, IDIV, CBW, CWD
Logic and shifts	AND, OR, XOR, NOT, TEST, SHL, SHR, SAR, ROL, ROR, RCL, RCR
Control transfer	JMP, Jcc (JE, JNE, JA, JB, JG, JL, ...), LOOP, LOOPE, LOOPNE, JCXZ, CALL, RET, INT
String	MOVSB/MOVSW, CMPSB, SCASB, LODSB, STOSB, REP prefixes, CLD, STD

APPENDIX B: MIPS / QtSPIM QUICK REFERENCE

B.1 Registers

Name	Number	Usage
$zero	0	Constant value 0
$at	1	Reserved for the assembler (pseudo-instructions)
$v0 – $v1	2–3	Values returned by procedures; syscall number in $v0
$a0 – $a3	4–7	Arguments to procedures and syscalls
$t0 – $t7	8–15	Temporaries (not preserved across calls)
$s0 – $s7	16–23	Saved registers (preserved across calls)
$t8 – $t9	24–25	More temporaries
$k0 – $k1	26–27	Reserved for the OS kernel
$gp	28	Global pointer
$sp	29	Stack pointer
$fp	30	Frame pointer
$ra	31	Return address (set by jal)

B.2 Frequently used instructions (* = pseudo-instruction)

Category	Instructions
Arithmetic	add, addu, addi, addiu, sub, subu, mult, div, mfhi, mflo, mul*
Logic and shifts	and, andi, or, ori, xor, nor, sll, srl, sra, sllv, srlv
Data transfer	lw, sw, lb, lbu, sb, lh, sh, lui, li*, la*, move*
Comparison	slt, slti, sltu
Branch and jump	beq, bne, blt*, bgt*, ble*, bge*, j, jal, jr
Other	syscall, nop

B.3 SPIM system calls

$v0	Service	Arguments	Result
1	print_int	$a0 = integer	–
4	print_string	$a0 = address of null-terminated string	–
5	read_int	–	$v0 = integer read
8	read_string	$a0 = buffer address, $a1 = buffer length	string stored in the buffer
9	sbrk (allocate memory)	$a0 = number of bytes	$v0 = address of the block
10	exit	–	program ends
11	print_char	$a0 = character	–
12	read_char	–	$v0 = character read

References

1. W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th ed., Pearson, 2019.
2. M. Morris Mano, *Computer System Architecture*, 3rd ed., Pearson Education, 2017.

3. D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface (MIPS Edition)*, Morgan Kaufmann.
4. Y. Yu and C. Marut, *Assembly Language Programming and Organization of the IBM PC*, McGraw-Hill.
5. J. R. Larus, “SPIM: A MIPS32 Simulator”, https://spimsimulator.sourceforge.net